



第四章

自顶向下的语法分析

重点：自顶向下分析的基本思想，预测分析器总体结构，预测分析表的构造，递归下降分析法基本思想，简单算术表达式的递归下降分析器。

难点：FIRST 和 FOLLOW 集的求法，对它们的理解以及在构造 LL(1)分析表时的使用。递归子程序法中如何体现分析的结果。



第4章 自顶向下的语法分析

4.1 语法分析概述

4.2 自顶向下的语法分析面临的问题

4.3 预测分析法

4.4 递归下降分析法

4.5 本章小结



语法分析的功能和位置

□**语法分析**(syntax analysis)是编译程序的核心部分，其任务是检查词法分析器输出的单词序列是否是源语言中的句子，亦即是否符合源语言的语法规则。

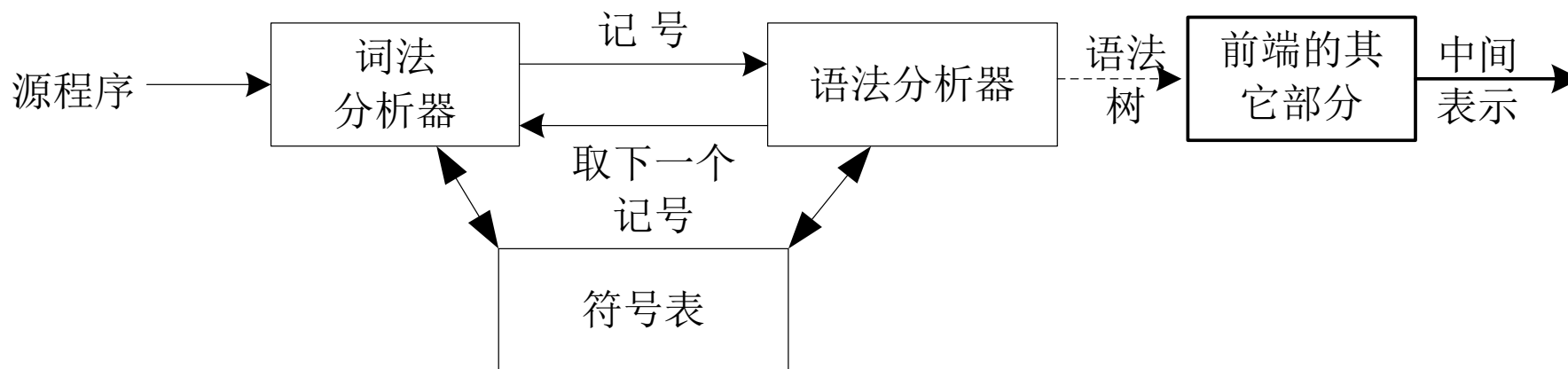


图4.1 语法分析器在编译器中的位置



4.1 语法分析概述



问题：解决确定性问题！

假定文法是压缩的：即删除了单位产生式和无用产生式。

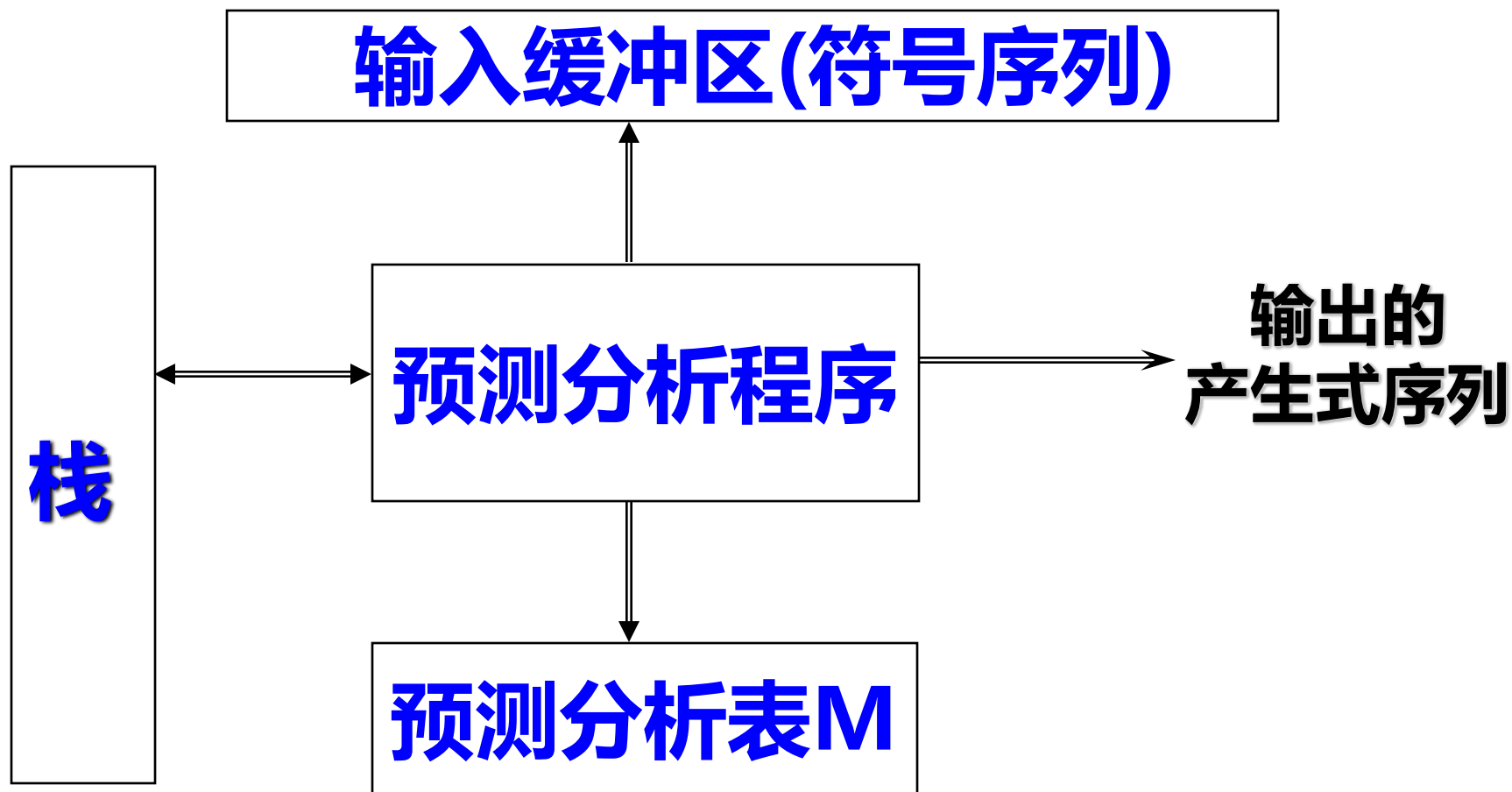


确定性是什么？预测分析法

- 系统维持一个分析表和一个分析栈，根据当前扫描到的符号，选择当前语法变量（处于栈顶）的候选式进行推导——希望找到相应输入符号串的最左推导。
- 一个通用的控制算法
- 一个分析栈，#为栈底符号
- 一个输入缓冲区，#为输入串结束符
- 一个统一形式的分析表M
 - 不同语言使用内容不同的分析表



确定性是什么？预测分析器的构成





对输入串id+id*id进行分析的过程

栈	输入缓冲区	输出
#E	id+id*id#	
#E'T	id+id*id#	$E \rightarrow TE'$
#E'T'F	id+id*id#	$T \rightarrow FT'$
#E'T'id	id+id*id#	$F \rightarrow id$
#E'T'	+id*id#	
#E'	+id*id#	$T' \rightarrow \epsilon$
#E'T+	+id*id#	$E' \rightarrow +TE'$
#E'T	id*id#	

非终结符	输入符号					
	id	+	*	(	)	#
E	$\rightarrow TE'$			$\rightarrow TE'$		
E'		$\rightarrow +TE'$			$\rightarrow \epsilon$	$\rightarrow \epsilon$
T	$\rightarrow FT'$			$\rightarrow FT'$		
T'		$\rightarrow \epsilon$	$\rightarrow *FT'$		$\rightarrow \epsilon$	$\rightarrow \epsilon$
F	$\rightarrow id$			$\rightarrow (E)$		



对输入串id+id*id进行分析的过程

#E'T	id*id#	
#E'T'F	id*id#	T→FT'
#E'T'id	id*id#	F→id
#E'T'	*id#	
#E'T'F*	*id#	T'→*FT'
#E'T'F	id#	
#E'T'id	id#	F→id
#E'T'	#	
#E'	#	T'→ε
#	#	E'→ε

非终结符	输入符号					
	id	+	*	(	)	#
E	→TE'			→TE'		
E'		→+TE'			→ε	→ε
T	→FT'			→FT'		
T'		→ε	→*FT'		→ε	→ε
F	→id			→(E)		

输出的产生式序列形成了最左推导对应的分析树

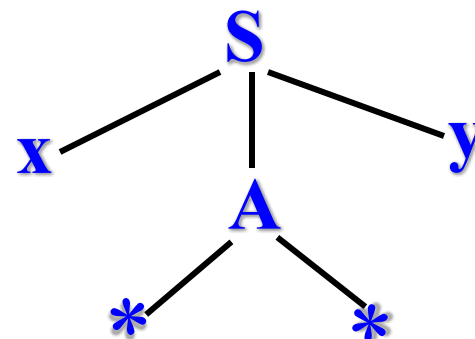
4.2 自顶向下的语法分析面临的问题

□ 自顶向下语法分析的基本思想

- 从文法的开始符号出发，寻求所给的输入符号串的一个最左推导。
- 从树根S开始，构造所给输入符号串的语法树

□ 例: 设有G: $S \rightarrow xAy$ $A \rightarrow **|*$, 输入串: $x**y$

$S \Rightarrow xAy$
 $\Rightarrow x**y$





4.2.1 自顶向下分析面临的问题

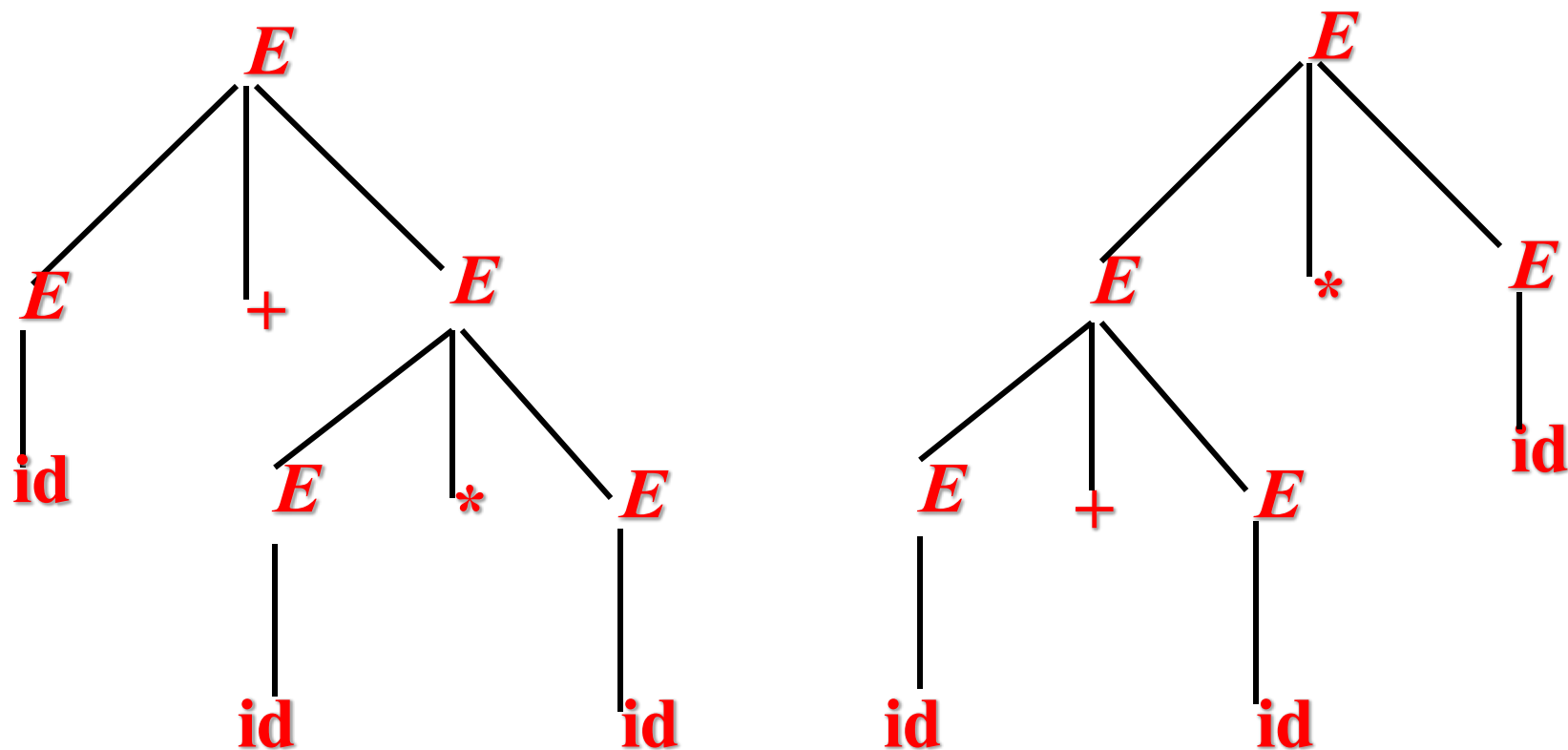
1、二义性问题

- 对于文法 G ，如果 $L(G)$ 中存在一个具有两棵或两棵以上分析树的句子，则称 G 是二义性的。也可以等价地说：如果 $L(G)$ 中存在一个具有两个或两个以上最左(或最右)推导的句子，则 G 是二义性文法。
- 如果一个文法 G 是二义性的，假设 $w \in L(G)$ 且 w 存在两个最左推导，则在对 w 进行自顶向下的语法分析时，语法分析程序将无法确定采用 w 的哪个最左推导。



4.2.1 自顶向下分析面临的问题

- 对同一句子存在两棵语法分析树
 - 在理论上不可判定

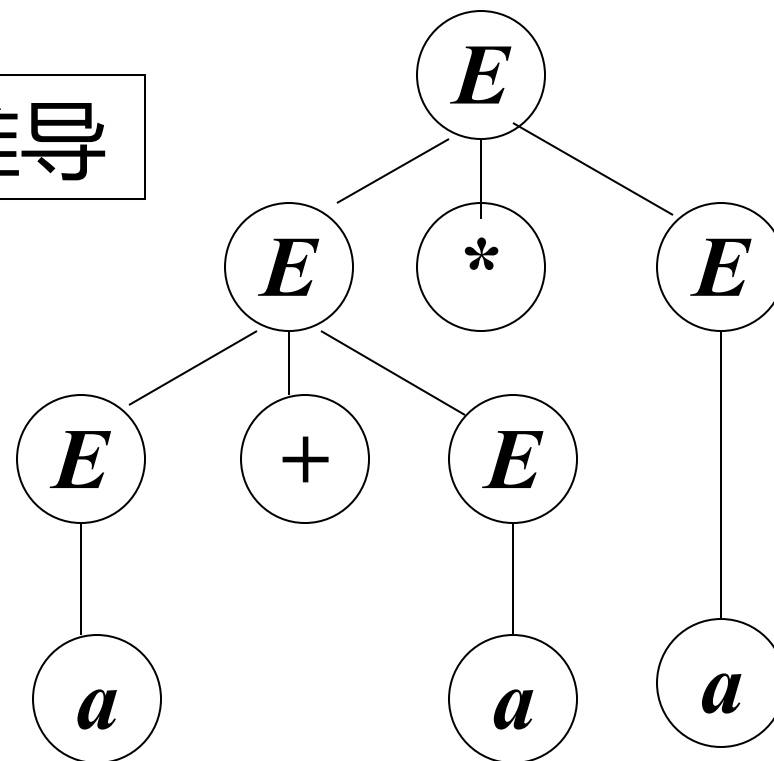
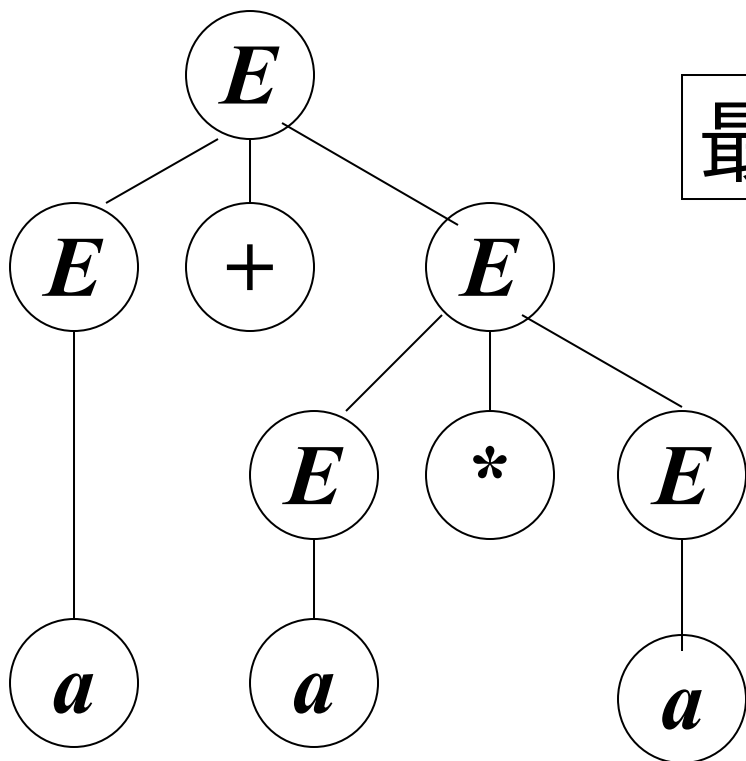




$$\begin{aligned} E &\Rightarrow E + E \Rightarrow a + E \\ &\Rightarrow a + E^* E \Rightarrow a + a^* E \\ &\Rightarrow a + a^* a \end{aligned}$$

$$\begin{aligned} E &\Rightarrow E^* E \Rightarrow E + E^* E \\ &\Rightarrow a + E^* E \Rightarrow a + a^* E \\ &\Rightarrow a + a^* a \end{aligned}$$

最左推导





4.2.1 自顶向下分析面临的问题

1、二义性问题

- 对于文法 G ，如果 $L(G)$ 中存在一个具有两棵或两棵以上分析树的句子，则称 G 是二义性的。也可以等价地说：如果 $L(G)$ 中存在一个具有两个或两个以上最左(或最右)推导的句子，则 G 是二义性文法。
- 如果一个文法 G 是二义性的，假设 $w \in L(G)$ 且 w 存在两个最左推导，则在对 w 进行自顶向下的语法分析时，语法分析程序将无法确定采用 w 的哪个最左推导。
- G_{exp} :
$$E \rightarrow E+T \mid E-T \mid T$$
$$T \rightarrow T * F \mid T / F \mid F$$
$$F \rightarrow F \uparrow P \mid P$$
$$P \rightarrow c \mid id \mid (E)$$

解决办法：改造文法，引入新的文法变量



4.2.1 自顶向下分析面临的问题

2、回溯问题

- 文法中每个语法变量A的产生式右部称为A的候选式，如果A有多个候选式存在公共前缀，则自顶向下的语法分析程序将无法根据当前输入符号准确地选择用于推导的产生式，只能试探。当试探不成功时就需要退回到上一步推导，看A是否还有其它的候选式，这就是回溯(backtracking)。
- Ge: $E \rightarrow T$ $E \rightarrow E+T$ $E \rightarrow E-T$ $T \rightarrow F$ $T \rightarrow T * F$ $T \rightarrow T / F$ $F \rightarrow (E)$ $F \rightarrow id$
- 例如：考虑为输入串id+id*id建立最左推导



4.2.1 自顶向下分析面临的问题

2、回溯问题 (id+id*id)

$$\square \quad E \Rightarrow T \quad (4.1)$$

$$\square \quad E \Rightarrow T \Rightarrow F \quad (4.2)$$

$$\square \quad E \Rightarrow T \Rightarrow F \Rightarrow (E) \quad (4.3)$$

$$\square \quad E \Rightarrow T \Rightarrow F \Rightarrow \text{id} \quad (4.4)$$

$$\square \quad E \Rightarrow T \Rightarrow T * F \quad (4.5)$$

$$\square \quad \dots\dots\dots$$

4.2.2节我们将采用提取左因子的方法来改造文法，以便减少推导过程中回溯现象的发生，当然，单纯通过提取左因子无法彻底避免回溯现象的发生。



4.2.1 自顶向下分析面临的问题

3、左递归引起的无穷推导问题

- 假设 A 是文法 G 的某个语法变量，如果存在推导 $A \Rightarrow^+ \alpha A \beta$ ，则称文法 G 是递归的，当 $\alpha = \varepsilon$ 时称之为**左递归**；如果 $A \Rightarrow^+ \alpha A \beta$ 至少需要两步推导，则称文法 G 是间接递归的，当 $\alpha = \varepsilon$ 时称之为**间接左递归**；如果文法 G 中存在形如 $A \rightarrow \alpha A \beta$ 的产生式，则称文法 G 是直接递归的，当 $\alpha = \varepsilon$ 时称之为**直接左递归**。
- $Ger: E \rightarrow T \quad E \rightarrow E+T \quad E \rightarrow E-T \quad T \rightarrow F \quad T \rightarrow T * F \quad T \rightarrow T / F \quad F \rightarrow (E) \quad F \rightarrow id$
- 考虑为输入串 $id+id*id$ 建立一个最左推导
- $E \Rightarrow E+T \Rightarrow E+T+T \Rightarrow E+T+T+T \Rightarrow \dots$

如果文法 G 是无二义的，则以下正确的是。

- | | |
|---|-------------------------------|
| A | 任何句子的最左推导和最右推导对应的语法树必定相同 |
| B | 任何句型的最左推导和最右推导对应的语法树必定相同 |
| C | 任何句子的最左推导和最右推导对应的语法树可能不同 |
| D | 任何句型的最左推导和最右推导对应的语法树可能不同 |
| E | 任何句子可能存在两个不同的最左推导，但它们对应的语法树相同 |
| F | 任何句型可能存在两个不同的最左推导，但它们对应的语法树相同 |



4.2.2 对上下文无关文法的改造

1、消除二义性

- 改造的方法就是通过引入新的语法变量等，使文法含有更多的信息。其实，许多二义性文法是由于概念不清，即语法变量的定义不明确导致的，此时通过引入新的语法变量即可消除文法的二义性。
- $\langle stmt \rangle \rightarrow \text{if } \langle expr \rangle \text{ then } \langle stmt \rangle$
- $\quad \quad \quad | \text{if } \langle expr \rangle \text{ then } \langle stmt \rangle \text{ else } \langle stmt \rangle$
- $\quad \quad \quad | \text{other} \quad \quad \quad (4.7)$
- 根据if语句中else与then配对情况将其分为配对的语句和不配对的语句两类。上述if语句的文法没有对这两个不同的概念加以区分，只是简单地将它们都定义为 $\langle stmt \rangle$ ，从而导致该文法是二义性的。



4.2.2 对上下文无关文法的改造

1、消除二义性

- 改造的方法就是通过引入新的语法变量等，使文法含有更多的信息。其实，许多二义性文法是由于概念不清，即语法变量的定义不明确导致的，此时通过引入新的语法变量即可消除文法的二义性。
- $\langle stmt \rangle \rightarrow \text{if } \langle expr \rangle \text{ then } \langle stmt \rangle$
- $\quad \quad \quad | \text{ if } \langle expr \rangle \text{ then } \langle stmt \rangle \text{ else } \langle stmt \rangle$
- $\quad \quad \quad | \text{ other} \quad \quad \quad (4.7)$
- 根据if语句中else与then配对情况将其分为配对的语句和不配对的语句两类。上述if语句的文法没有对这两个不同的概念加以区分，只是简单地将它们都定义为 $\langle stmt \rangle$ ，从而导致该文法是二义性的。
- $a=5; c=4; b=6;$
- If $a < 6$ then if $c < 3$ then $b=5$ else $b=8$? Output: $b=6$
- If $a < 6$ then if $c < 3$ then $b=5$ else $b=8$? Output: $b=8$



4.2.2 对上下文无关文法的改造

引入语法变量 $\langle \text{unmatched_stmt} \rangle$ 来表示不配对语句,

$\langle \text{matched_stmt} \rangle$ 表示配对语句

- $\langle \text{stmt} \rangle \rightarrow \langle \text{matched_stmt} \rangle$
| $\langle \text{unmatched_stmt} \rangle$
- $\langle \text{matched_stmt} \rangle \rightarrow \text{if } \langle \text{expr} \rangle \text{ then } \langle \text{matched_stmt} \rangle \text{ else } \langle \text{matched_stmt} \rangle$
| other
- $\langle \text{unmatched_stmt} \rangle \rightarrow \text{if } \langle \text{expr} \rangle \text{ then } \langle \text{stmt} \rangle$
| $\text{if } \langle \text{expr} \rangle \text{ then } \langle \text{matched_stmt} \rangle \text{ else } \langle \text{unmatched_stmt} \rangle$



4.2.2 对上下文无关文法的改造

2、消除左递归

- 直接左递归的消除(转换为右递归)
- 引入新的变量 A' ，将左递归产生式 $A \rightarrow A\alpha|\beta$ 替换为 $A \rightarrow \beta A' \quad A' \rightarrow \alpha A' |\varepsilon$
- $E \rightarrow E+T|T \quad T \rightarrow T*F|F \quad F \rightarrow (E)|id$ 替换为：
- $E \rightarrow TE' \quad E' \rightarrow +TE'|\varepsilon \quad T \rightarrow FT' \quad T' \rightarrow *FT'|\varepsilon \quad F \rightarrow (E)|id$

□ 一般地，假设文法 G 中的语法变量 A 的所有产生式如下：

$$A \rightarrow A\alpha_1|A\alpha_2|\dots|A\alpha_n|\beta_1|\beta_2|\dots|\beta_m$$

其中， $\beta_i(i=1,2,\dots,m)$ 不以 A 打头。则用如下的产生式代替 A 的所有产生式即可消除其直接左递归： $A \rightarrow \beta_1 A'|\beta_2 A'|\dots|\beta_m A' \quad A' \rightarrow \alpha_1 A'|\alpha_2 A'|\dots|\alpha_n A'|\varepsilon$



4.2.2 对上下文无关文法的改造

算法4.1 消除左递归。

输入：不含循环推导和 ε -产生式的文法 G ;

输出：与 G 等价的无左递归文法;

步骤：

1. 将 G 的所有语法变量排序(编号), 假设排序后的语法变量记为 $A_1, A_2, \dots, A_n$;
2. for $i \leftarrow 1$ to n {
3. for $j \leftarrow 1$ to $i-1$ {
4. 用产生式 $A_i \rightarrow \alpha_1 \beta | \alpha_2 \beta | \dots | \alpha_k \beta$ 代替每个形如 $A_i \rightarrow A_j \beta$ 的产生式,
 其中, $A_j \rightarrow \alpha_1 | \alpha_2 | \dots | \alpha_k$ 是所有的当前 A_j 产生式;
5. }
6. 消除 A_i 产生式中的所有直接左递归
7. }



4.2.2 对上下文无关文法的改造

3.提取左因子

- 对每个语法变量 A ，找出它的两个或更多候选式的最长公共前缀 α 。如果 $\alpha \neq \varepsilon$ ，则用下面的产生式替换所有的 A 产生式 $A \rightarrow \alpha\beta_1 | \alpha\beta_2 | \dots | \alpha\beta_n | \gamma_1 | \gamma_2 | \dots | \gamma_n$ ，其中 $\gamma_1, \gamma_2, \dots, \gamma_n$ 表示所有不以 α 开头的候选式：

$$A \rightarrow \alpha A' | \gamma_1 | \gamma_2 | \dots | \gamma_n$$

$$A' \rightarrow \beta_1 | \beta_2 | \dots | \beta_n$$

- 其中， A' 是新引入的语法变量。反复应用上述变换，直到任意语法变量都没有两个候选式具有公共前缀为止。请同学们自行给出这个变换的算法。

采用自上而下分析，不必()。

- ☐ A 消除回溯
- ☐ B 消除左递归
- ☐ C 提取公共左因子
- ☒ D 消除右递归

提交



4.2.3 LL(1)文法

问题：什么样的文法对其句子才能进行确定的自顶向下分析？

- 确定的自顶向下分析首先从文法的开始符号出发，每一步推导都根据当前句型的最左语法变量 A 和当前输入符号 a ，选择 A 的某个候选式 α 来替换 A ，并使得从 α 推导出的第一个终结符恰好是 a 。
- 当 A 有多个候选式时，当前选中的候选式必须是惟一的。
- 第一个终结符是指符号串的第一个符号，并且是终结符号，可以称为**首终结符号**。在自顶向下的分析中，它对选取候选式具有重要的作用。为此引入首符号集的概念。



4.2.3 LL(1)文法

1. 假设 α 是文法 $G=(V, T, P, S)$ 的符号串, 即 $\alpha \in (V \cup T)^*$

从 α 推导出的串的首符号集记作 $\text{FIRST}(\alpha)$:

$$\text{FIRST}(\alpha) = \{a | \alpha \xRightarrow{*} a\beta, a \in T, \beta \in (V \cup T)^*\}.$$

2. 如果 $\alpha \xRightarrow{*} \varepsilon$, 则 $\varepsilon \in \text{FIRST}(\alpha)$ 。

3. 如果文法 G 中的所有 A 产生式为 $A \rightarrow \alpha_1 | \alpha_2 | \dots | \alpha_m$, 且

$$\varepsilon \notin \text{FIRST}(\alpha_1) \cup \text{FIRST}(\alpha_2) \cup \dots \cup \text{FIRST}(\alpha_m)$$

且对 $\forall i, j, 1 \leq i, j \leq m; i \neq j$, 均有

$$\text{FIRST}(\alpha_i) \cap \text{FIRST}(\alpha_j) = \emptyset \text{ 成立,}$$

则可以对 G 的句子进行确定的自顶向下分析



4.2.3 LL(1)文法

如果存在 $A \rightarrow \varepsilon$ 这样的产生式，则需定义 $\text{FOLLOW}(A)$

$\forall A \in V$ 定义 A 的后续符号集为：

1. $\text{FOLLOW}(A) = \{b \mid S \xRightarrow{*} \alpha A b \beta, b \in T, \alpha, \beta \in (V \cup T)^*\}$
2. 如果 A 是某个句型的最右符号，则将结束符 $\#$ 添加到 $\text{FOLLOW}(A)$ 中
3. 如果 $\alpha_j \xRightarrow{*} \varepsilon$ ，则如果对 $\forall i (1 \leq i \leq m; i \neq j)$ ， $\text{FIRST}(\alpha_i) \cap \text{FOLLOW}(A) = \emptyset$ 均成立，则可以对 G 的句子进行确定的自顶向下分析



4.2.3 LL(1)文法

如果 G 的任意两个具有相同左部的产生式 $A \rightarrow \alpha | \beta$ 满足下列条件:

1. 如果 α 、 β 均不能推导出 ε , 则 $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$;
2. α 和 β 至多有一个能推导出 ε ;
3. 如果 $\beta \xRightarrow{*} \varepsilon$, 且 $\text{FIRST}(\alpha) \cap \text{FOLLOW}(A) = \emptyset$

则称 G 为LL(1)文法

第一个 L 代表从左向右扫描输入符号串, 第二个 L 代表产生最左推导, 1代表在分析过程中执行每步推导都要向前查看一个输入符号



LL(1)文法的判定

算法4.2 计算FIRST(X)。

输入：文法 $G=(V, T, P, S)$, $X \in (V \cup T)$;

输出：FIRST(X);

步骤：

1. FIRST(X) = $\emptyset$;
2. if ($X \in T$) then FIRST(X) := {X} ;
3. if $X \in V$ then
 begin
4. if ($X \rightarrow \varepsilon \notin P$) then FIRST(X) := FIRST(X) $\cup$ {a | $X \rightarrow a \dots \in P$ } ;
5. if ($X \rightarrow \varepsilon \in P$) then FIRST(X) := FIRST(X) $\cup$ {a | $X \rightarrow a \dots \in P$ } $\cup$ { ε } ;
- end
6. 对 $\forall X \in V$, 重复如下的过程7-10, 直到所有FIRST集不变为止。
7. if ($X \rightarrow Y \dots \in P$ and $Y \in V$) then FIRST(X) := FIRST(X) $\cup$ (FIRST(Y) - { ε });
8. if ($X \rightarrow Y_1 \dots Y_n \in P$ and $Y_1 \dots Y_{i-1} \overset{*}{\Rightarrow} \varepsilon$) then
9. for $k=2$ to i do FIRST(X) := FIRST(X) $\cup$ (FIRST(Y_k) - { ε });
10. if $Y_1 \dots Y_n \overset{*}{\Rightarrow} \varepsilon$ then FIRST(X) := FIRST(X) $\cup$ { ε } ;



LL(1)文法的判定

算法4.3 计算FIRST(α)。

输入：文法 $G=(V, T, P, S)$, $\alpha \in (V \cup T)^*$, $\alpha = X_1 \dots X_n$;

输出：FIRST(α);

步骤：

1. 计算FIRST(X_1);
2. FIRST(α) := FIRST(X_1) - $\{\epsilon\}$;
3. $k := 1$;
4. while ($\epsilon \in \text{FIRST}(X_k)$ and $k < n$) do begin
5. FIRST(α) := FIRST(α) $\cup$ (FIRST(X_{k+1}) - $\{\epsilon\}$);
6. $k := k + 1$ end
7. if ($k = n$ and $\epsilon \in \text{FIRST}(X_k)$) then FIRST(α) := FIRST(α) $\cup$ $\{\epsilon\}$;

例 表达式文法的语法符号的FIRST 集

$\text{FIRST}(F) = \{ (, \text{id} \}$

$\text{FIRST}(T) = \text{FIRST}(F) = \{ (, \text{id} \}$

$\text{FIRST}(E) = \text{FIRST}(T) = \{ (, \text{id} \}$

$\text{FIRST}(E') = \{ +, \varepsilon \}$

$\text{FIRST}(T') = \{ *, \varepsilon \}$

$\text{FIRST}(+) = \{ + \}, \text{FIRST}(*) = \{ * \}$

$\text{FIRST}(()) = \{ (\}$

$\text{FIRST}()) = \{) \}$

$\text{FIRST}(\text{id}) = \{ \text{id} \}$

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \varepsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \varepsilon$

$F \rightarrow (E) \mid \text{id}$



LL(1)文法的判定

算法4.4 计算FOLLOW集。

输入：文法 $G=(V, T, P, S)$, $A \in V$;

输出：FOLLOW(A);

步骤：

1. 对 $\forall X \in V$, FOLLOW(X) := $\emptyset$;
2. FOLLOW(S) := $\{\#\}$, #为句子的结束符;
3. 对 $\forall X \in V$, 重复下面的第4步到第5步, 直到所有FOLLOW集不变为止。
4. 若 $A \rightarrow \alpha B \beta \in P$, 则FOLLOW(B) := FOLLOW(B) $\cup$ FIRST(β) - $\{\varepsilon\}$;
5. 若 $A \rightarrow \alpha B$ 或 $A \rightarrow \alpha B \beta \in P$, 且 $\beta \xRightarrow{*} \varepsilon$, $A \neq B$, 则
FOLLOW(B) := FOLLOW(B) $\cup$ FOLLOW(A);

例 表达式文法的语法变量的 FOLLOW 集

$\text{FOLLOW}(S) := \{\#\}$, #为句子的结束符;

若 $A \rightarrow \alpha B \beta \in P$, 则

$\text{FOLLOW}(B) := \text{FOLLOW}(B) \cup \text{FIRST}(\beta) - \{\epsilon\}$;

$E \rightarrow TE' \quad E' \rightarrow +TE' | \epsilon$

$T \rightarrow FT' \quad T' \rightarrow *FT' | \epsilon$

$F \rightarrow (E)id$

$\text{FOLLOW}(E) = \{\#, \text{)}\}$

$\text{FIRST}(F) = \{ (, id \}$

$\text{FIRST}(T) = \text{FIRST}(F) = \{ (, id \}$

$\text{FIRST}(E) = \text{FIRST}(T) = \{ (, id \}$

$\text{FIRST}(E') = \{ +, \epsilon \}$

$\text{FIRST}(T') = \{ *, \epsilon \}$

例 表达式文法的语法变量的 FOLLOW 集

4. 若 $A \rightarrow \alpha B \beta \in P$, 则

$$\text{FOLLOW}(B) := \text{FOLLOW}(B) \cup \text{FIRST}(\beta) - \{\varepsilon\};$$

5. 若 $A \rightarrow \alpha B$ 或 $A \rightarrow \alpha B \beta \in P$, 且 $\beta \Rightarrow \varepsilon$, $A \neq B$, 则

$$\text{FOLLOW}(B) := \text{FOLLOW}(B) \cup \text{FOLLOW}(A);$$

$$E \rightarrow TE' \quad E' \rightarrow +TE' | \varepsilon$$

$$T \rightarrow FT' \quad T' \rightarrow *FT' | \varepsilon$$

$$F \rightarrow (E) | \text{id}$$

$$\text{FOLLOW}(E) = \{ \#,) \}$$

$$\text{FOLLOW}(E') = \text{FOLLOW}(E) = \{ \#,) \}$$

$$\text{FIRST}(F) = \{ (, \text{id} \}$$

$$\text{FIRST}(T) = \text{FIRST}(F) = \{ (, \text{id} \}$$

$$\text{FIRST}(E) = \text{FIRST}(T) = \{ (, \text{id} \}$$

$$\text{FIRST}(E') = \{ +, \varepsilon \}$$

$$\text{FIRST}(T') = \{ *, \varepsilon \}$$

例 表达式文法的语法变量的 FOLLOW 集

4. 若 $A \rightarrow \alpha B \beta \in P$, 则

$$\text{FOLLOW}(B) := \text{FOLLOW}(B) \cup \text{FIRST}(\beta) - \{\epsilon\};$$

5. 若 $A \rightarrow \alpha B$ 或 $A \rightarrow \alpha B \beta \in P$, 且 $\beta \Rightarrow \epsilon$, $A \neq B$, 则

$$\text{FOLLOW}(B) := \text{FOLLOW}(B) \cup \text{FOLLOW}(A);$$

$$E \rightarrow TE' \quad E' \rightarrow +TE' | \epsilon$$

$$T \rightarrow FT' \quad T' \rightarrow *FT' | \epsilon$$

$$F \rightarrow (E) | \text{id}$$

$$\text{FOLLOW}(E) = \{ \#,) \}$$

$$\text{FOLLOW}(E') = \text{FOLLOW}(E) = \{ \#,) \}$$

$$\text{FOLLOW}(T) = \{ \text{FIRST}(E') - \{\epsilon\} \} \cup \text{FOLLOW}(E) \cup \text{FOLLOW}(E') = \{ +,), \# \}$$

$$\text{FIRST}(F) = \{ (, \text{id} \}$$

$$\text{FIRST}(T) = \text{FIRST}(F) = \{ (, \text{id} \}$$

$$\text{FIRST}(E) = \text{FIRST}(T) = \{ (, \text{id} \}$$

$$\text{FIRST}(E') = \{ +, \epsilon \}$$

$$\text{FIRST}(T') = \{ *, \epsilon \}$$

例 表达式文法的语法变量的 FOLLOW 集

4. 若 $A \rightarrow \alpha B \beta \in P$, 则

$$\text{FOLLOW}(B) := \text{FOLLOW}(B) \cup \text{FIRST}(\beta) - \{\epsilon\};$$

5. 若 $A \rightarrow \alpha B$ 或 $A \rightarrow \alpha B \beta \in P$, 且 $\beta \Rightarrow \epsilon$, $A \neq B$, 则

$$\text{FOLLOW}(B) := \text{FOLLOW}(B) \cup \text{FOLLOW}(A);$$

$$E \rightarrow TE' \quad E' \rightarrow +TE' | \epsilon$$

$$T \rightarrow FT' \quad T' \rightarrow *FT' | \epsilon$$

$$F \rightarrow (E) | \text{id}$$

$$\text{FOLLOW}(E) = \{ \#,) \}$$

$$\text{FOLLOW}(E') = \text{FOLLOW}(E) = \{ \#,) \}$$

$$\text{FOLLOW}(T) = \{ \text{FIRST}(E') - \{\epsilon\} \} \cup \text{FOLLOW}(E) \cup \text{FOLLOW}(E') = \{ +,), \# \}$$

$$\text{FIRST}(F) = \{ (, \text{id} \}$$

$$\text{FIRST}(T) = \text{FIRST}(F) = \{ (, \text{id} \}$$

$$\text{FIRST}(E) = \text{FIRST}(T) = \{ (, \text{id} \}$$

$$\text{FIRST}(E') = \{ +, \epsilon \}$$

$$\text{FIRST}(T') = \{ *, \epsilon \}$$

例 表达式文法的语法变量的 FOLLOW 集

4. 若 $A \rightarrow \alpha B \beta \in P$, 则

$$\text{FOLLOW}(B) := \text{FOLLOW}(B) \cup \text{FIRST}(\beta) - \{\epsilon\};$$

5. 若 $A \rightarrow \alpha B$ 或 $A \rightarrow \alpha B \beta \in P$, 且 $\beta \Rightarrow \epsilon$, $A \neq B$, 则

$$\text{FOLLOW}(B) := \text{FOLLOW}(B) \cup \text{FOLLOW}(A);$$

$$E \rightarrow TE' \quad E' \rightarrow +TE' | \epsilon$$

$$T \rightarrow FT' \quad T' \rightarrow *FT' | \epsilon$$

$$F \rightarrow (E) | \text{id}$$

$$\text{FOLLOW}(E) = \{ \#,) \}$$

$$\text{FOLLOW}(E') = \text{FOLLOW}(E) = \{ \#,) \}$$

$$\text{FOLLOW}(T) = \{ \text{FIRST}(E') - \{\epsilon\} \} \cup \text{FOLLOW}(E) \cup \text{FOLLOW}(E') = \{ +,), \# \}$$

$$\text{FIRST}(F) = \{ (, \text{id} \}$$

$$\text{FIRST}(T) = \text{FIRST}(F) = \{ (, \text{id} \}$$

$$\text{FIRST}(E) = \text{FIRST}(T) = \{ (, \text{id} \}$$

$$\text{FIRST}(E') = \{ +, \epsilon \}$$

$$\text{FIRST}(T') = \{ *, \epsilon \}$$

例 表达式文法的语法变量的 FOLLOW 集

4. 若 $A \rightarrow \alpha B \beta \in P$, 则

$$\text{FOLLOW}(B) := \text{FOLLOW}(B) \cup \text{FIRST}(\beta) - \{\epsilon\};$$

5. 若 $A \rightarrow \alpha B$ 或 $A \rightarrow \alpha B \beta \in P$, 且 $\beta \Rightarrow \epsilon$, $A \neq B$, 则

$$\text{FOLLOW}(B) := \text{FOLLOW}(B) \cup \text{FOLLOW}(A);$$

$$E \rightarrow TE' \quad E' \rightarrow +TE' | \epsilon$$

$$T \rightarrow FT' \quad T' \rightarrow *FT' | \epsilon$$

$$F \rightarrow (E) | \text{id}$$

$$\text{FOLLOW}(E) = \{ \#,) \}$$

$$\text{FOLLOW}(E') = \text{FOLLOW}(E) = \{ \#,) \}$$

$$\text{FOLLOW}(T) = \{ \text{FIRST}(E') - \{\epsilon\} \} \cup \text{FOLLOW}(E) \cup \text{FOLLOW}(E') = \{ +,), \# \}$$

$$\text{FOLLOW}(T') = \text{FOLLOW}(T) = \{ +,), \# \}$$

$$\text{FIRST}(F) = \{ (, \text{id} \}$$

$$\text{FIRST}(T) = \text{FIRST}(F) = \{ (, \text{id} \}$$

$$\text{FIRST}(E) = \text{FIRST}(T) = \{ (, \text{id} \}$$

$$\text{FIRST}(E') = \{ +, \epsilon \}$$

$$\text{FIRST}(T') = \{ *, \epsilon \}$$

例 表达式文法的语法变量的 FOLLOW 集

4. 若 $A \rightarrow \alpha B \beta \in P$, 则

$$\text{FOLLOW}(B) := \text{FOLLOW}(B) \cup \text{FIRST}(\beta) - \{\epsilon\};$$

5. 若 $A \rightarrow \alpha B$ 或 $A \rightarrow \alpha B \beta \in P$, 且 $\beta \Rightarrow \epsilon$, $A \neq B$, 则

$$\text{FOLLOW}(B) := \text{FOLLOW}(B) \cup \text{FOLLOW}(A);$$

$$E \rightarrow TE' \quad E' \rightarrow +TE' | \epsilon$$

$$T \rightarrow FT' \quad T' \rightarrow *FT' | \epsilon$$

$$F \rightarrow (E) | \text{id}$$

$$\text{FOLLOW}(E) = \{ \#,) \}$$

$$\text{FOLLOW}(E') = \text{FOLLOW}(E) = \{ \#,) \}$$

$$\text{FOLLOW}(T) = \{ \text{FIRST}(E') - \{\epsilon\} \} \cup \text{FOLLOW}(E) \cup \text{FOLLOW}(E') = \{ +,), \# \}$$

$$\text{FOLLOW}(T') = \text{FOLLOW}(T) = \{ +,), \# \}$$

$$\text{FOLLOW}(F) = \{ \text{FIRST}(T') - \{\epsilon\} \} \cup \text{FOLLOW}(T) \cup \text{FOLLOW}(T') = \{ *, +,), \# \}$$

$$\text{FIRST}(F) = \{ (, \text{id} \}$$

$$\text{FIRST}(T) = \text{FIRST}(F) = \{ (, \text{id} \}$$

$$\text{FIRST}(E) = \text{FIRST}(T) = \{ (, \text{id} \}$$

$$\text{FIRST}(E') = \{ +, \epsilon \}$$

$$\text{FIRST}(T') = \{ *, \epsilon \}$$

例 表达式文法的语法变量的 FOLLOW 集

4. 若 $A \rightarrow \alpha B \beta \in P$, 则

$$\text{FOLLOW}(B) := \text{FOLLOW}(B) \cup \text{FIRST}(\beta) - \{\epsilon\};$$

5. 若 $A \rightarrow \alpha B$ 或 $A \rightarrow \alpha B \beta \in P$, 且 $\beta \Rightarrow \epsilon$, $A \neq B$, 则

$$\text{FOLLOW}(B) := \text{FOLLOW}(B) \cup \text{FOLLOW}(A);$$

$$E \rightarrow TE' \quad E' \rightarrow +TE' | \epsilon$$

$$T \rightarrow FT' \quad T' \rightarrow *FT' | \epsilon$$

$$F \rightarrow (E) | \text{id}$$

$$\text{FOLLOW}(E) = \{ \#,) \}$$

$$\text{FOLLOW}(E') = \text{FOLLOW}(E) = \{ \#,) \}$$

$$\text{FOLLOW}(T) = \{ \text{FIRST}(E') - \{\epsilon\} \} \cup \text{FOLLOW}(E) \cup \text{FOLLOW}(E') = \{ +,), \# \}$$

$$\text{FOLLOW}(T') = \text{FOLLOW}(T) = \{ +,), \# \}$$

$$\text{FOLLOW}(F) = \{ \text{FIRST}(T') - \{\epsilon\} \} \cup \text{FOLLOW}(T) \cup \text{FOLLOW}(T') = \{ *, +,), \# \}$$

$$\text{FIRST}(F) = \{ (, \text{id} \}$$

$$\text{FIRST}(T) = \text{FIRST}(F) = \{ (, \text{id} \}$$

$$\text{FIRST}(E) = \text{FIRST}(T) = \{ (, \text{id} \}$$

$$\text{FIRST}(E') = \{ +, \epsilon \}$$

$$\text{FIRST}(T') = \{ *, \epsilon \}$$

例 表达式文法的语法变量的 FOLLOW 集

4. 若 $A \rightarrow \alpha B \beta \in P$, 则

$$\text{FOLLOW}(B) := \text{FOLLOW}(B) \cup \text{FIRST}(\beta) - \{\epsilon\};$$

5. 若 $A \rightarrow \alpha B$ 或 $A \rightarrow \alpha B \beta \in P$, 且 $\beta \Rightarrow \epsilon$, $A \neq B$, 则

$$\text{FOLLOW}(B) := \text{FOLLOW}(B) \cup \text{FOLLOW}(A);$$

$$E \rightarrow TE' \quad E' \rightarrow +TE' | \epsilon$$

$$T \rightarrow FT' \quad T' \rightarrow *FT' | \epsilon$$

$$F \rightarrow (E) | \text{id}$$

$$\text{FOLLOW}(E) = \{ \#,) \}$$

$$\text{FOLLOW}(E') = \text{FOLLOW}(E) = \{ \#,) \}$$

$$\text{FOLLOW}(T) = \{ \text{FIRST}(E') - \{\epsilon\} \} \cup \text{FOLLOW}(E) \cup \text{FOLLOW}(E') = \{ +,), \# \}$$

$$\text{FOLLOW}(T') = \text{FOLLOW}(T) = \{ +,), \# \}$$

$$\text{FOLLOW}(F) = \{ \text{FIRST}(T') - \{\epsilon\} \} \cup \text{FOLLOW}(T) \cup \text{FOLLOW}(T') = \{ *, +,), \# \}$$

$$\text{FIRST}(F) = \{ (, \text{id} \}$$

$$\text{FIRST}(T) = \text{FIRST}(F) = \{ (, \text{id} \}$$

$$\text{FIRST}(E) = \text{FIRST}(T) = \{ (, \text{id} \}$$

$$\text{FIRST}(E') = \{ +, \epsilon \}$$

$$\text{FIRST}(T') = \{ *, \epsilon \}$$

已知文法 $G[S]$:

$S \rightarrow eT \mid RT \quad T \rightarrow DR \mid \varepsilon \quad R \rightarrow dR \mid \varepsilon \quad D \rightarrow a \mid bd$

求 $FOLLOW(D)$ 。

A $\{d, e\}$

B $\{d, \varepsilon\}$

C $\{d, \#\}$

D $\{a, d\}$

提交



解析

已知文法G[S]:

$S \rightarrow eT \mid RT$ $T \rightarrow DR \mid \varepsilon$ $R \rightarrow dR \mid \varepsilon$ $D \rightarrow a \mid bd$

求FOLLOW(D)。

$\{d, \#\}$ First(R)

$\{d, \#\}$ Follow(T)=Follow(S)



表达式文法是 LL(1) 文法

□ $E \rightarrow T E'$

□ $E' \rightarrow + T E' \mid \varepsilon$

□ $T \rightarrow F T'$

□ $T' \rightarrow * F T' \mid \varepsilon$

□ $F \rightarrow (E) \mid \text{id}$

考察

□ E' : $+$ 不在 $\text{FOLLOW}(E') = \{), \# \}$

□ T' : $*$ 不在 $\text{FOLLOW}(T') = \{ +,), \# \}$

□ F : $($ 和 id 不同

如果 G 的任意两个具有相同左部的产生式 $A \rightarrow \alpha \mid \beta$ 满足下列条件:

1. 如果 α 、 β 均不能推导出 ε , 则 $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$;
2. α 和 β 至多有一个能推导出 ε ;
3. 如果 $\beta \Rightarrow \varepsilon$, 且 $\text{FIRST}(\alpha) \cap \text{FOLLOW}(A) = \emptyset$

则称 G 为 **LL(1) 文法**



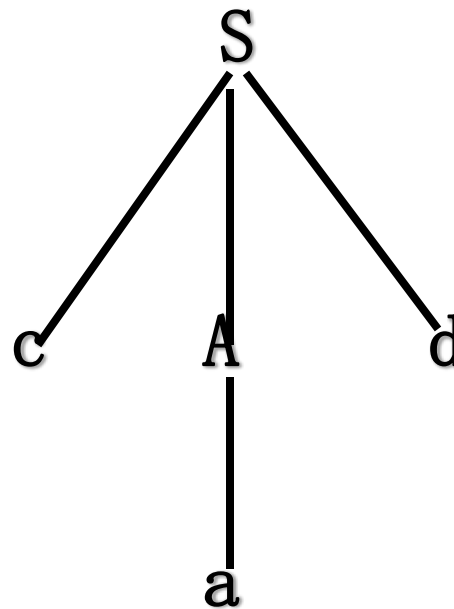
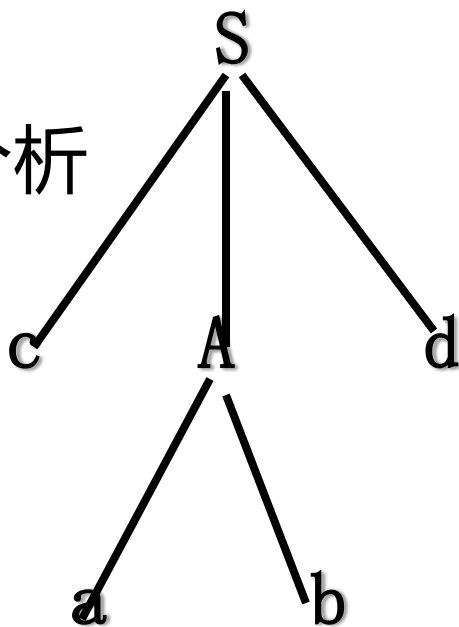
非 LL(1)文法的不确定性

例 对文法

□ $S \rightarrow cAd$

□ $A \rightarrow ab|a$

输入 cad 的分析





不确定性的解决方法

1) 采用回溯算法

- 过于复杂，效率低下

2) 改写文法

- 将非LL(1)文法改写为等价的LL(1)文法

□无法改写时：

- 增加其它的判别因素
- 文法过于复杂，无法用自顶向下方法处理

()文法一定不是LL(1)的。

- ☐ A 递归
- ☐ B 右递归
- ☐ C 2型
- ☒ D 含有公共左因子的

LL（1）分析法中“1”的含义是在输入串中查看一个输入符号，其目的是（ ）。

- ☐ A 确定最左推导
- ☐ B 确定句柄
- ☒ C 确定使用哪一个产生式进行展开
- ☐ D 确定是否推导

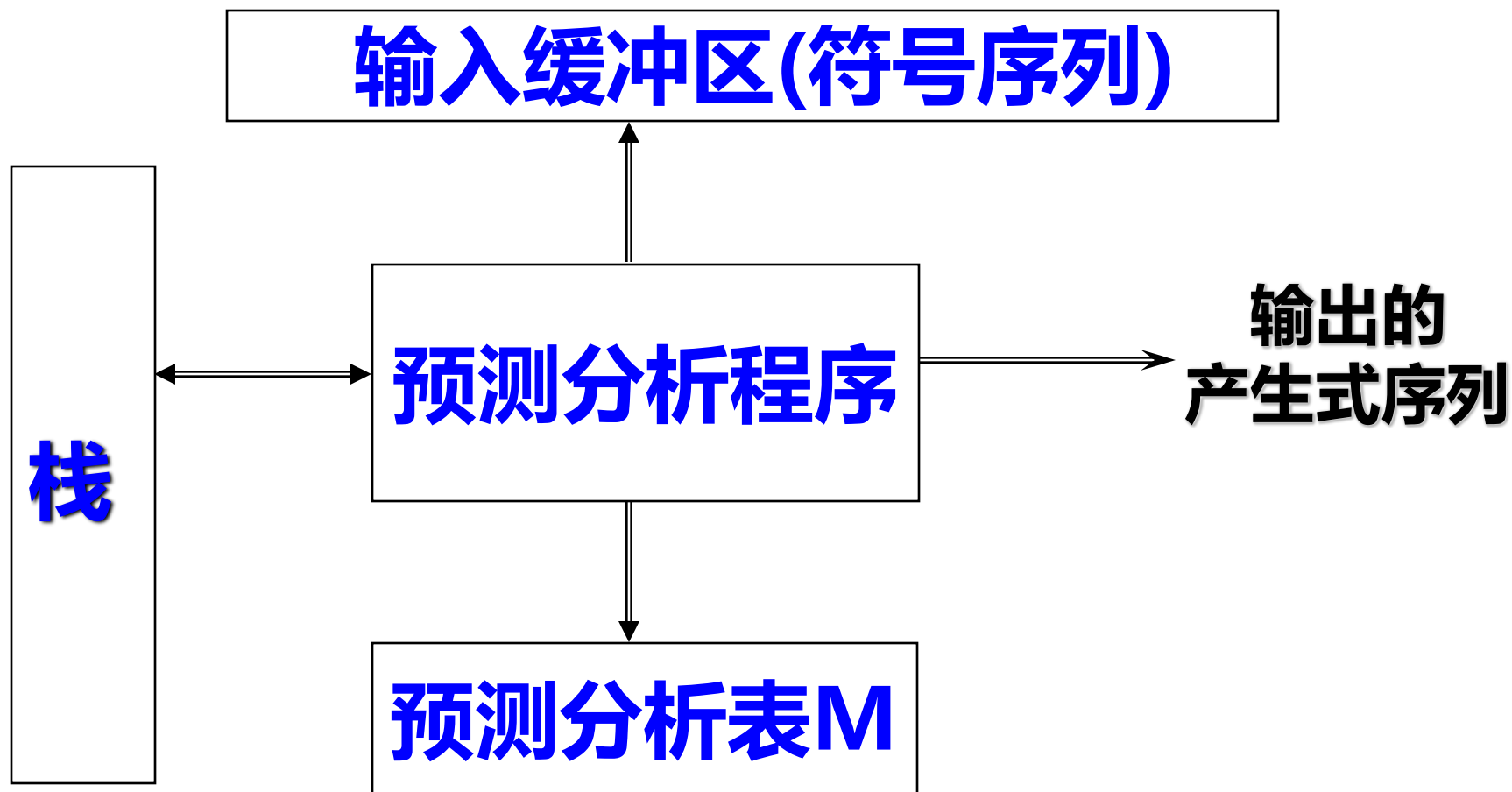


4.3 预测分析法

- 系统维持一个分析表和一个分析栈，根据当前扫描到的符号，选择当前语法变量（处于栈顶）的候选式进行推导——希望找到相应输入符号串的最左推导。
- 一个通用的控制算法
- 一个分析栈，#为栈底符号
- 一个输入缓冲区，#为输入串结束符
- 一个统一形式的分析表M
 - 不同语言使用内容不同的分析表

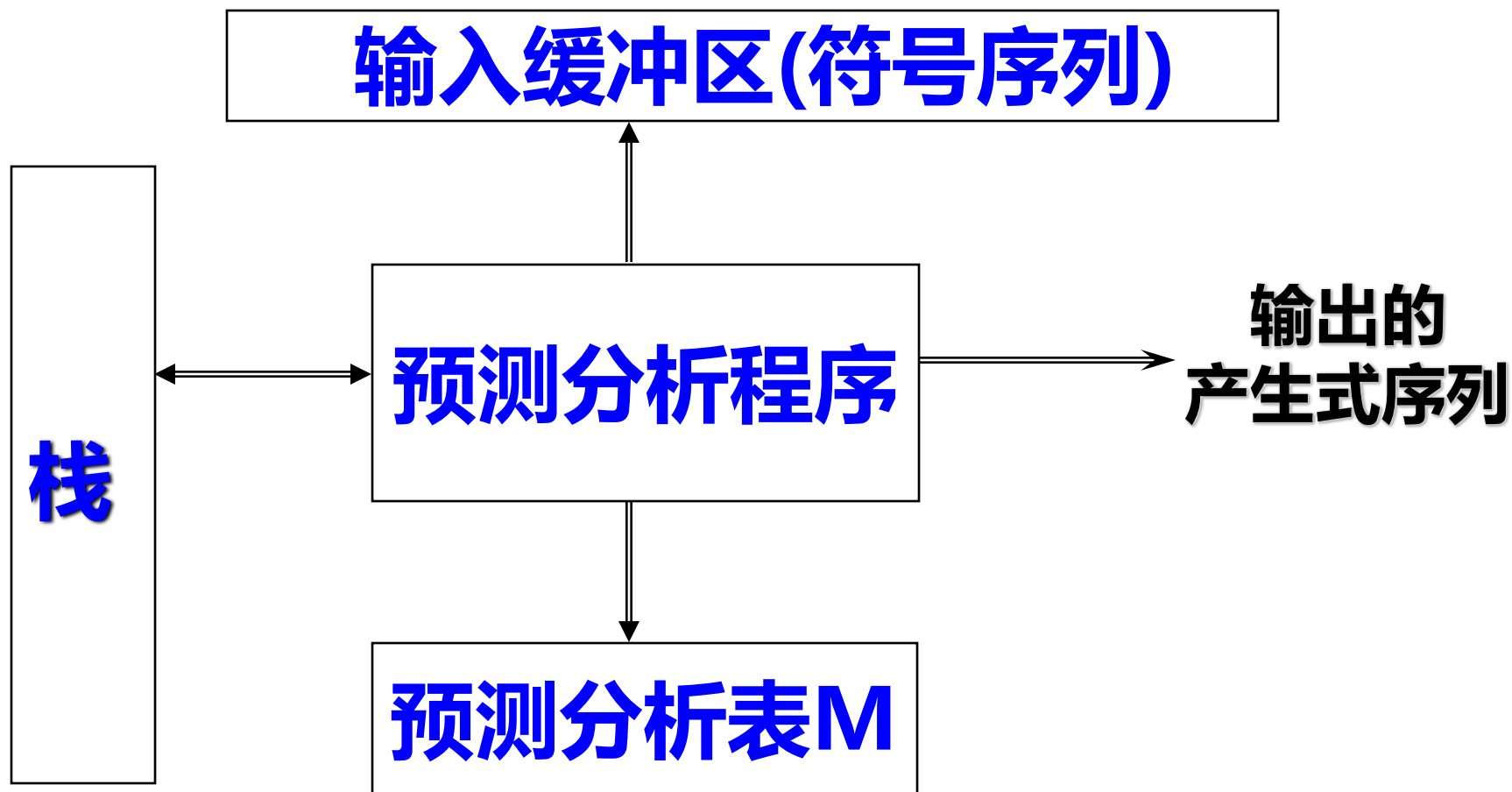


4.3.1 预测分析器的构成



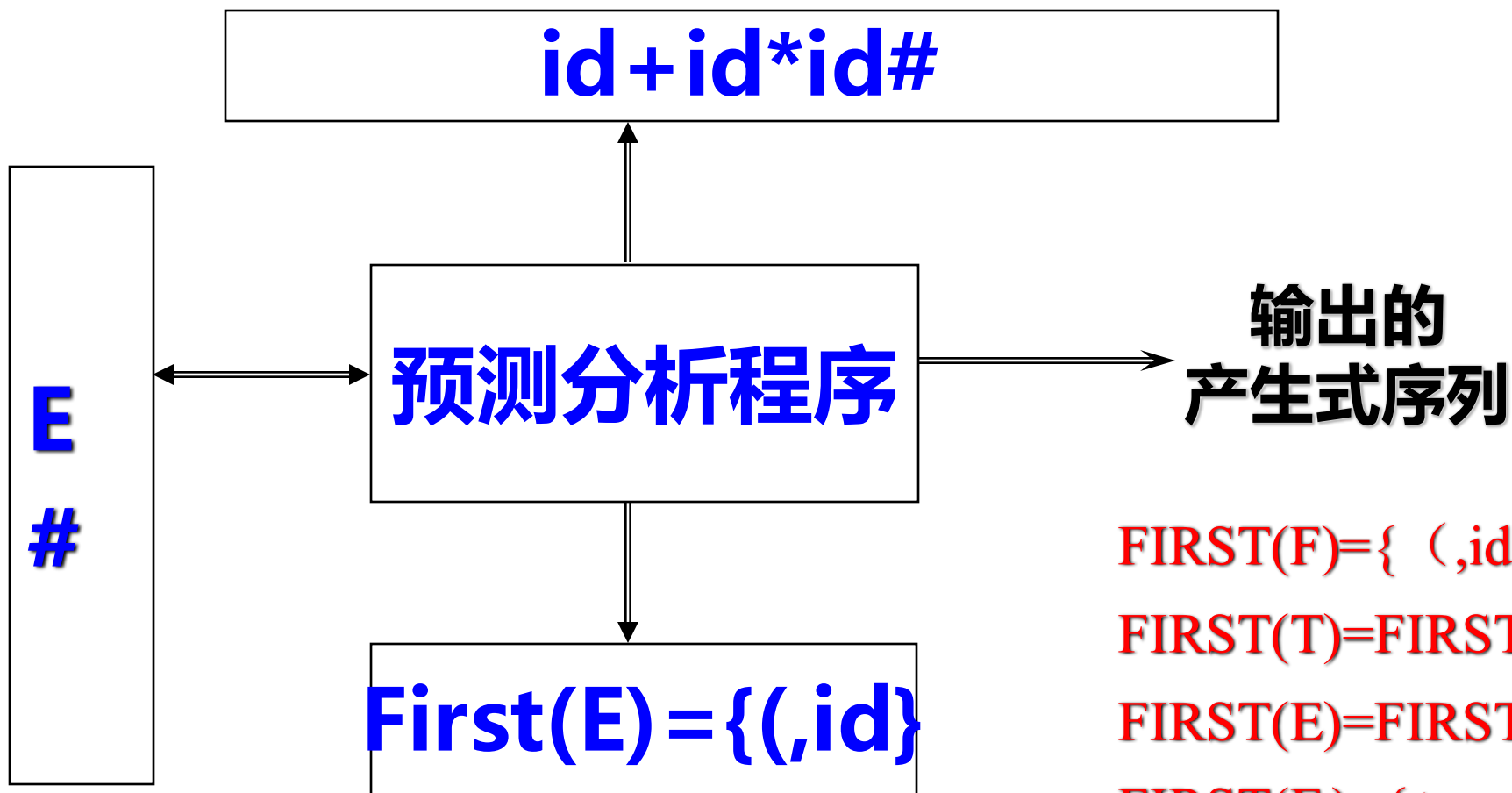


4.3.1 预测分析器的构成



4.3.1 预测分析器的构成

$E \rightarrow TE'$ $E' \rightarrow +TE' | \varepsilon$ $T \rightarrow FT'$ $T' \rightarrow *FT' | \varepsilon$ $F \rightarrow (E) | id$



$FIRST(F) = \{ (, id \}$

$FIRST(T) = FIRST(F) = \{ (, id \}$

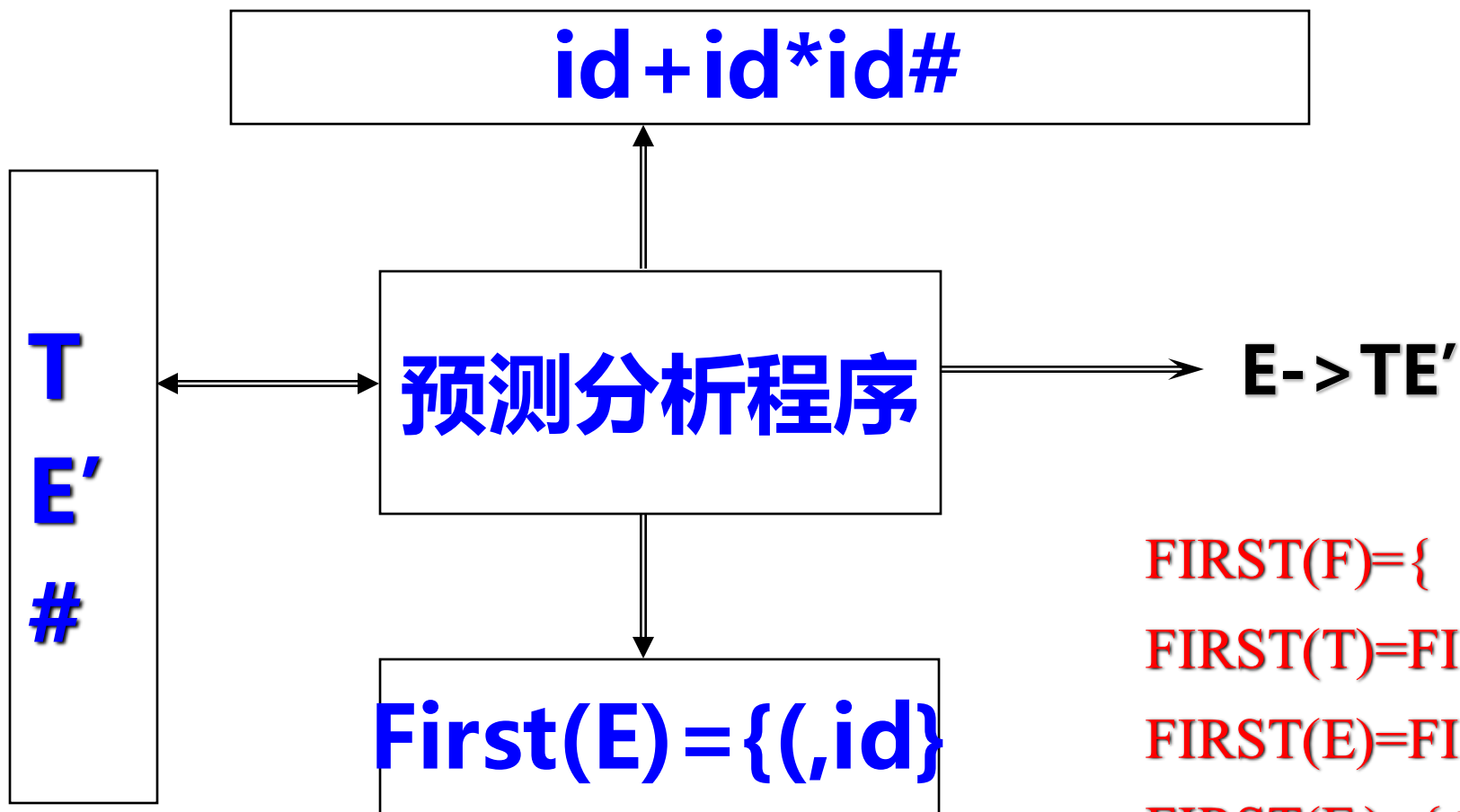
$FIRST(E) = FIRST(T) = \{ (, id \}$

$FIRST(E') = \{ +, \varepsilon \}$

$FIRST(T') = \{ *, \varepsilon \}$

4.3.1 预测分析器的构成

$E \rightarrow TE'$ $E' \rightarrow +TE' | \varepsilon$ $T \rightarrow FT'$ $T' \rightarrow *FT' | \varepsilon$ $F \rightarrow (E) | id$



$FIRST(F) = \{ (, id \}$

$FIRST(T) = FIRST(F) = \{ (, id \}$

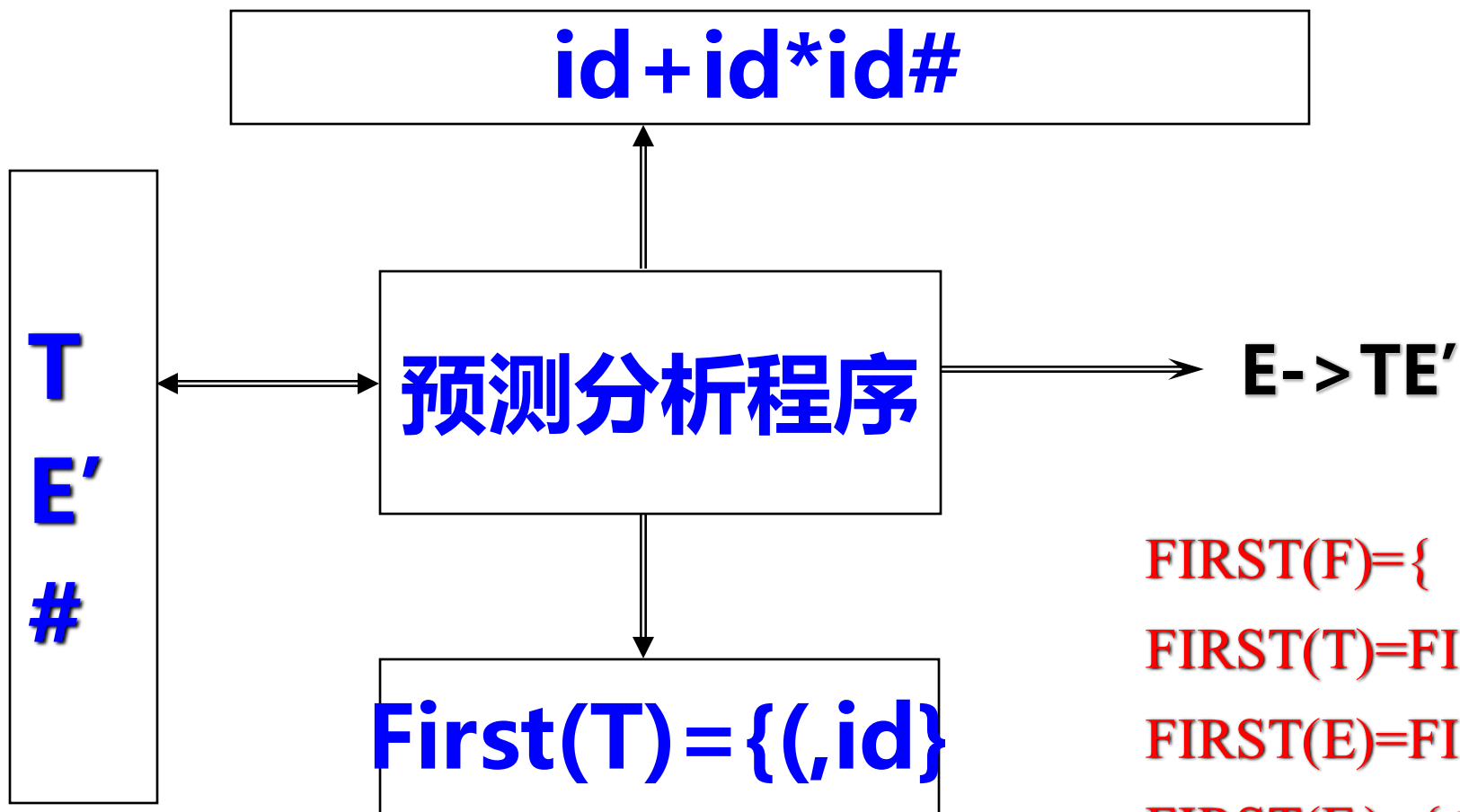
$FIRST(E) = FIRST(T) = \{ (, id \}$

$FIRST(E') = \{ +, \varepsilon \}$

$FIRST(T') = \{ *, \varepsilon \}$

4.3.1 预测分析器的构成

$E \rightarrow TE'$ $E' \rightarrow +TE' | \varepsilon$ $T \rightarrow FT'$ $T' \rightarrow *FT' | \varepsilon$ $F \rightarrow (E) | id$



$FIRST(F) = \{ (, id \}$

$FIRST(T) = FIRST(F) = \{ (, id \}$

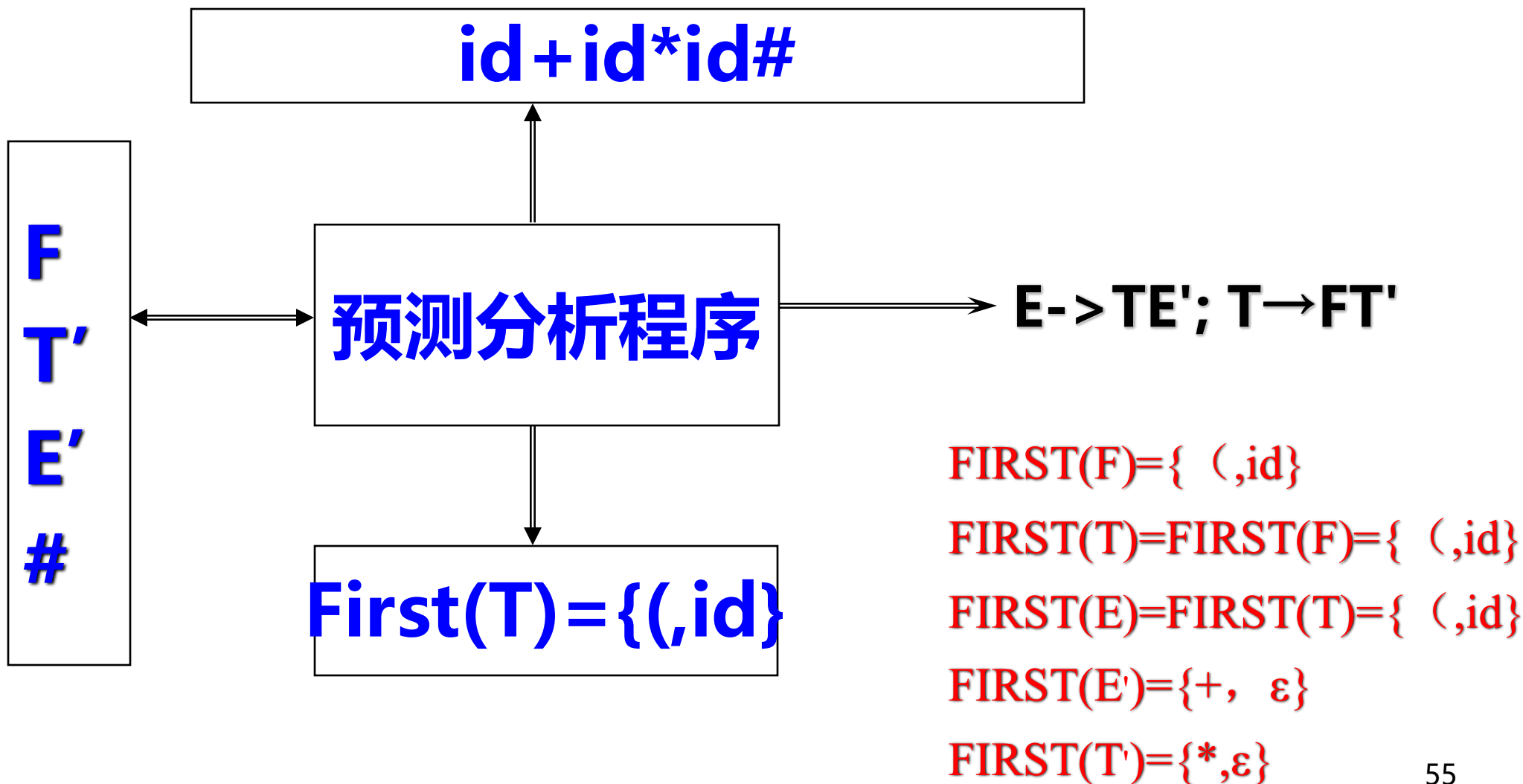
$FIRST(E) = FIRST(T) = \{ (, id \}$

$FIRST(E') = \{ +, \varepsilon \}$

$FIRST(T') = \{ *, \varepsilon \}$

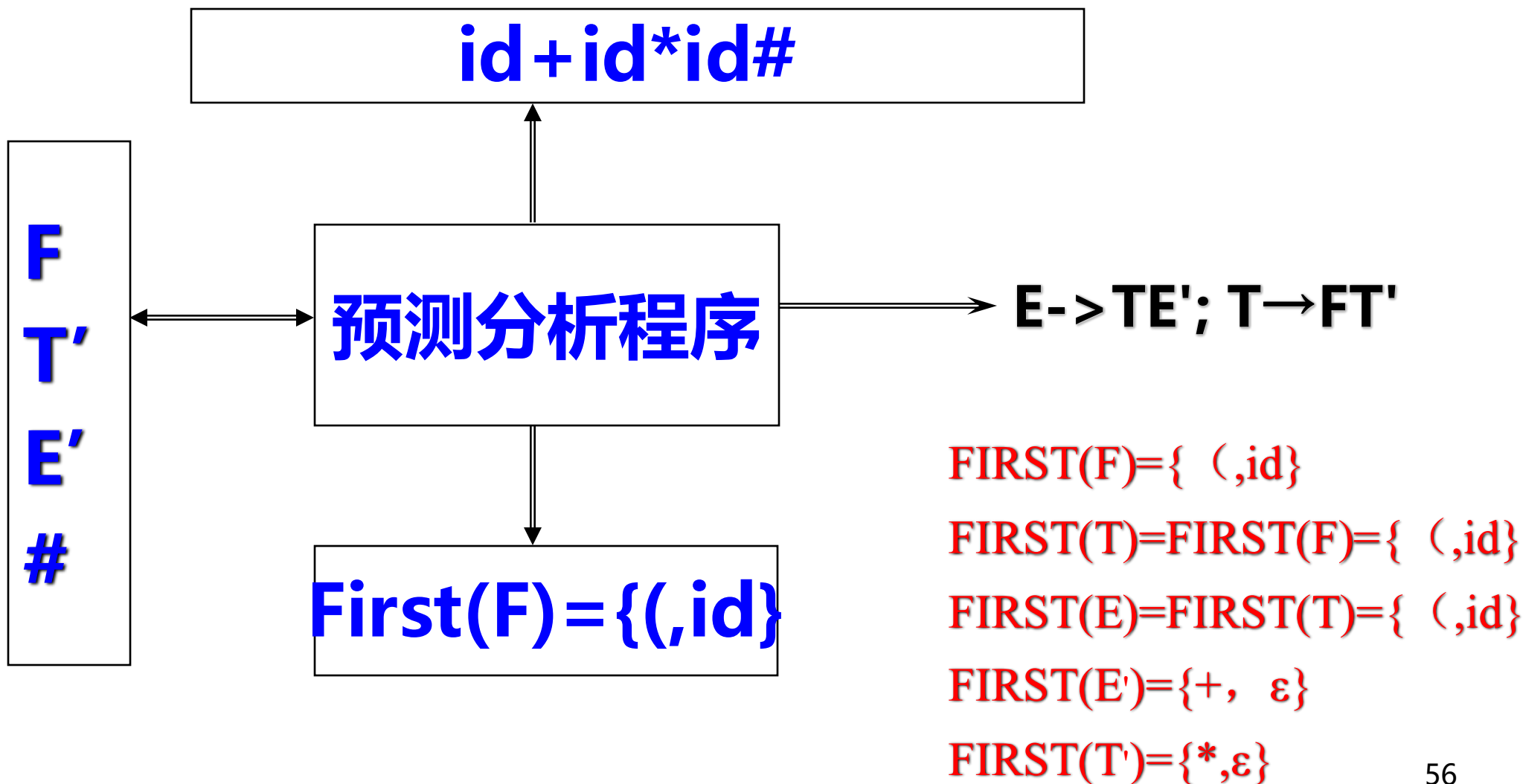
4.3.1 预测分析器的构成

$E \rightarrow TE'$ $E' \rightarrow +TE' | \varepsilon$ $T \rightarrow FT'$ $T' \rightarrow *FT' | \varepsilon$ $F \rightarrow (E) | id$



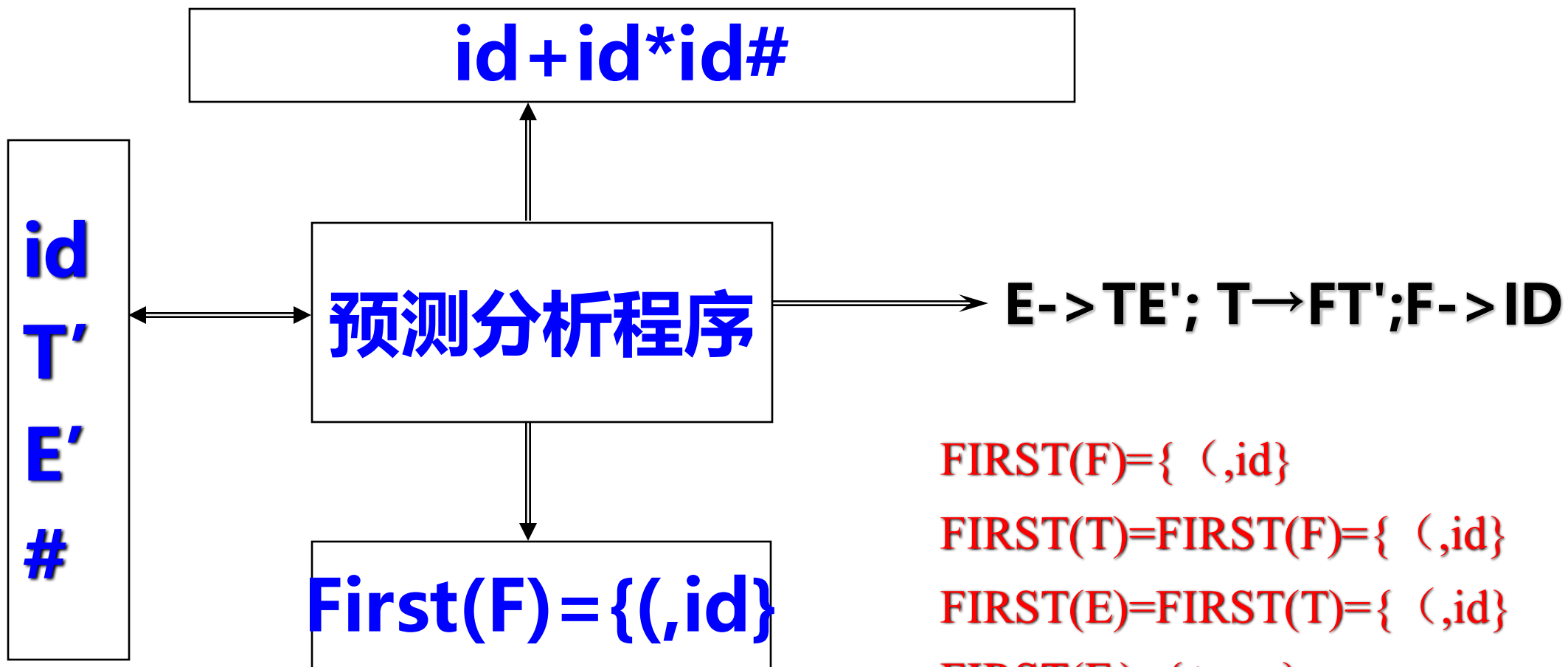
4.3.1 预测分析器的构成

$E \rightarrow TE'$ $E' \rightarrow +TE' | \varepsilon$ $T \rightarrow FT'$ $T' \rightarrow *FT' | \varepsilon$ $F \rightarrow (E) | id$



4.3.1 预测分析器的构成

$E \rightarrow TE'$ $E' \rightarrow +TE' | \varepsilon$ $T \rightarrow FT'$ $T' \rightarrow *FT' | \varepsilon$ $F \rightarrow (E) | id$



$FIRST(F) = \{ (, id \}$

$FIRST(T) = FIRST(F) = \{ (, id \}$

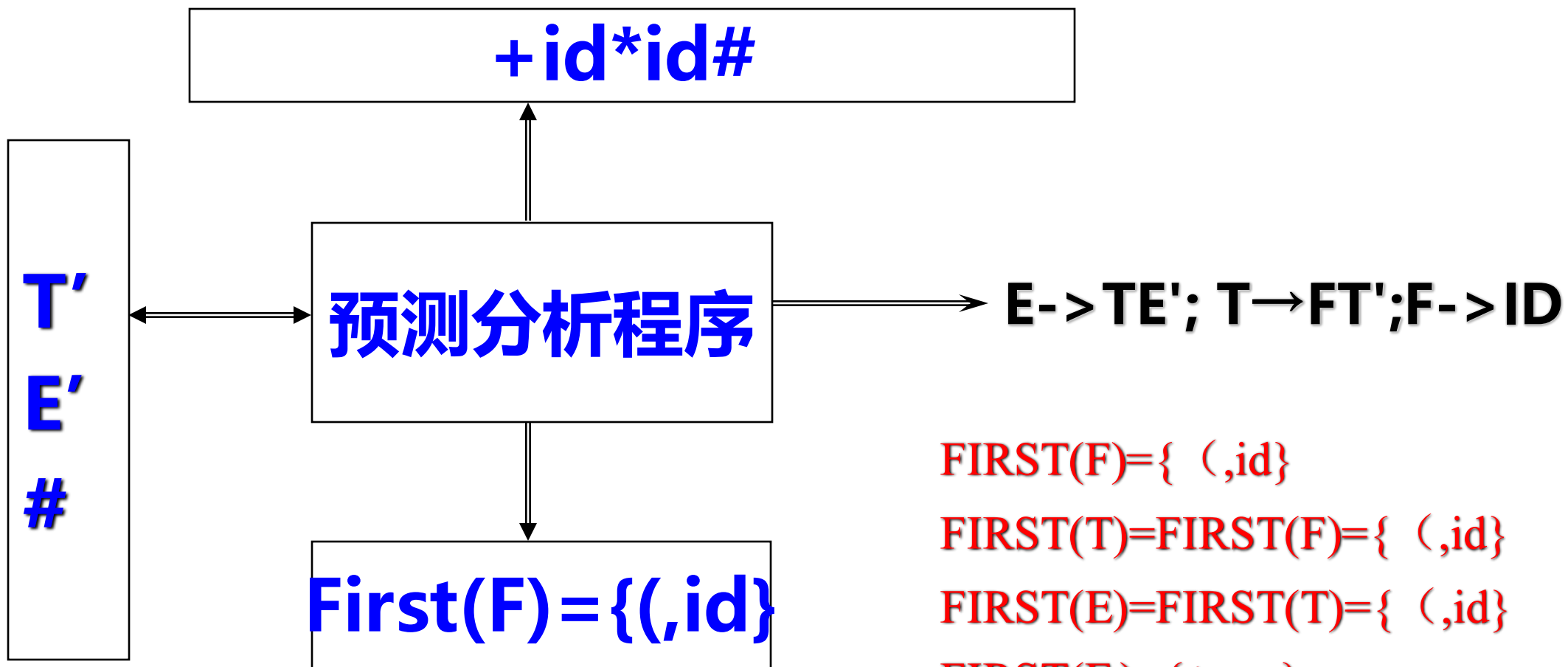
$FIRST(E) = FIRST(T) = \{ (, id \}$

$FIRST(E') = \{ +, \varepsilon \}$

$FIRST(T') = \{ *, \varepsilon \}$

4.3.1 预测分析器的构成

$E \rightarrow TE'$ $E' \rightarrow +TE' | \varepsilon$ $T \rightarrow FT'$ $T' \rightarrow *FT' | \varepsilon$ $F \rightarrow (E) | id$



$FIRST(F) = \{ (, id \}$

$FIRST(T) = FIRST(F) = \{ (, id \}$

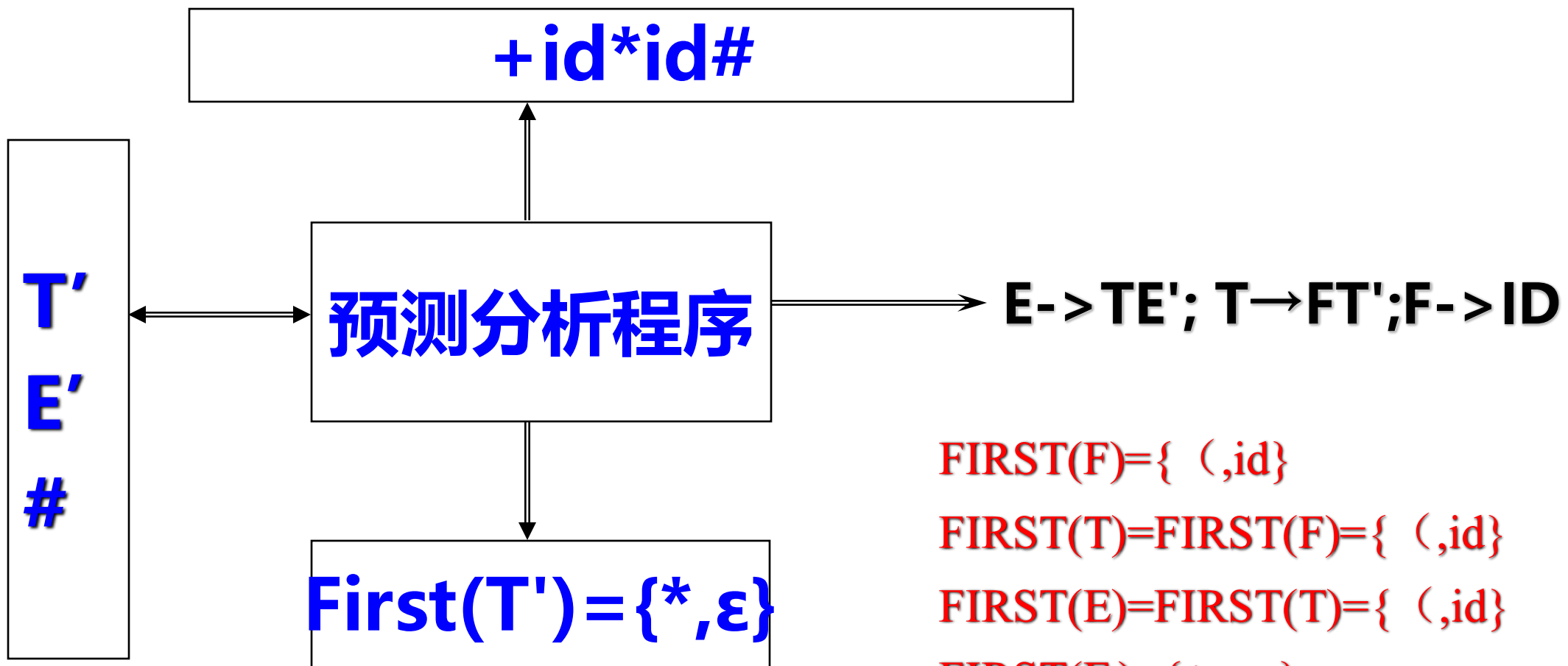
$FIRST(E) = FIRST(T) = \{ (, id \}$

$FIRST(E') = \{ +, \varepsilon \}$

$FIRST(T') = \{ *, \varepsilon \}$

4.3.1 预测分析器的构成

$E \rightarrow TE'$ $E' \rightarrow +TE' | \varepsilon$ $T \rightarrow FT'$ $T' \rightarrow *FT' | \varepsilon$ $F \rightarrow (E) | id$



$\text{FIRST}(F) = \{ (, id \}$

$\text{FIRST}(T) = \text{FIRST}(F) = \{ (, id \}$

$\text{FIRST}(E) = \text{FIRST}(T) = \{ (, id \}$

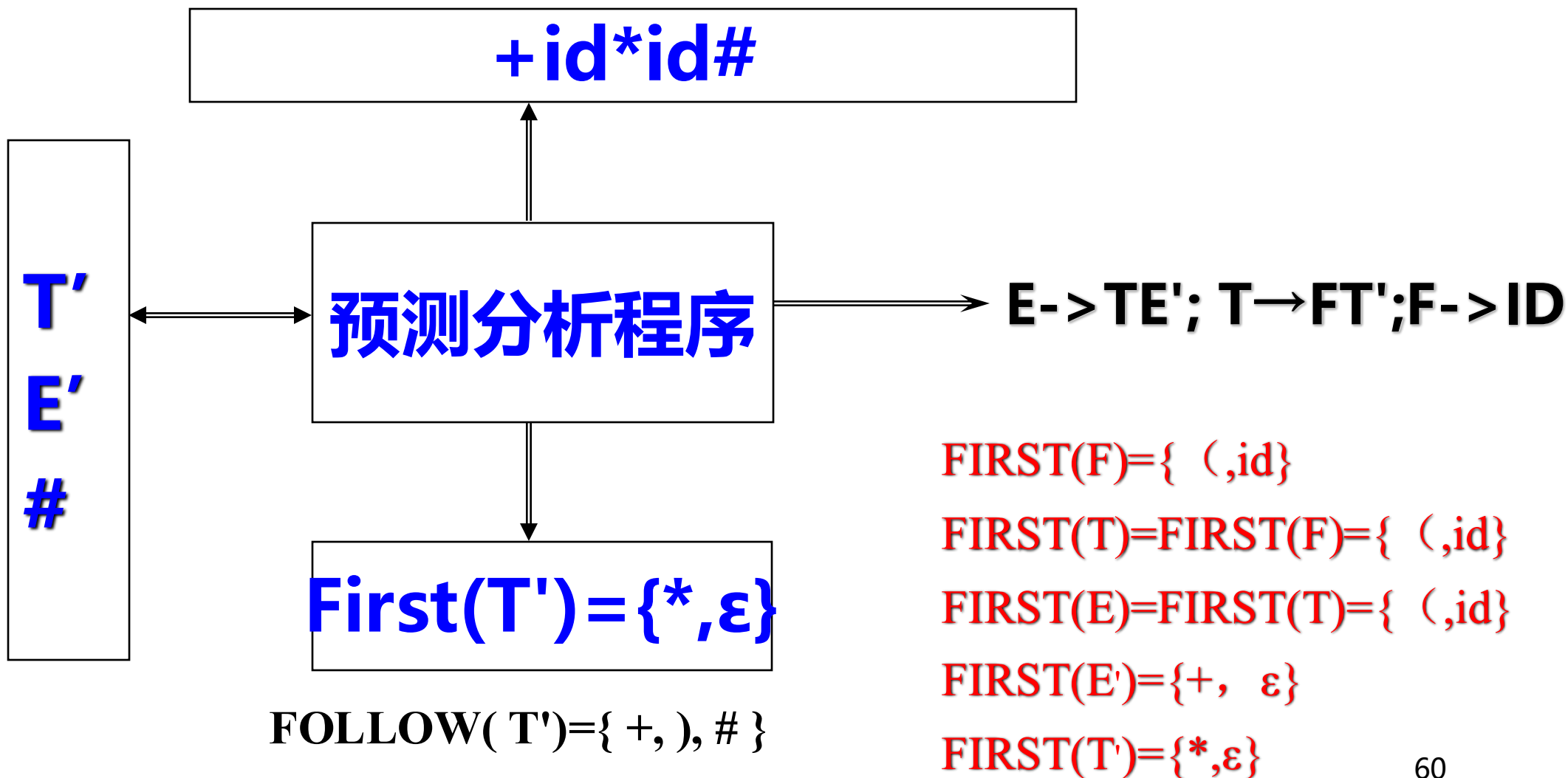
$\text{FIRST}(E') = \{ +, \varepsilon \}$

$\text{FIRST}(T') = \{ *, \varepsilon \}$



4.3.1 预测分析器的构成

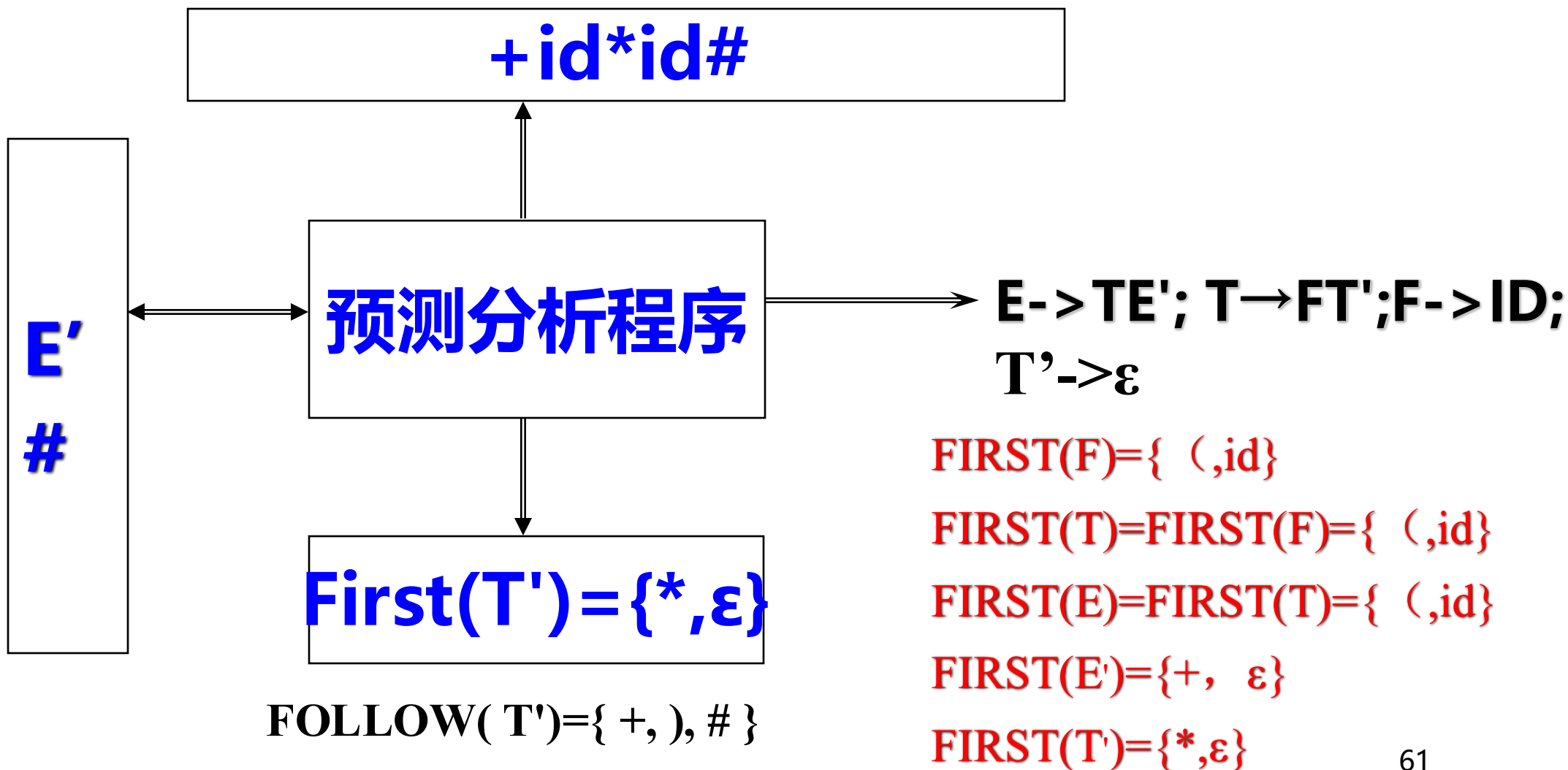
$E \rightarrow TE'$ $E' \rightarrow +TE' | \varepsilon$ $T \rightarrow FT'$ $T' \rightarrow *FT' | \varepsilon$ $F \rightarrow (E) | id$





4.3.1 预测分析器的构成

$E \rightarrow TE'$ $E' \rightarrow +TE' | \varepsilon$ $T \rightarrow FT'$ $T' \rightarrow *FT' | \varepsilon$ $F \rightarrow (E) | id$





系统的执行与特点

- 在系统启动时，输入指针指向输入串的第一个字符，分析栈中存放着栈底符号#和文法的开始符号。
- 根据栈顶符号A和读入的符号a，查看分析表M, 以决定相应的动作。
- 优点：
 - 1) 效率高
 - 2) 便于维护、自动生成
- 关键——分析表M的构造



预测分析程序的总控程序

算法4.5 预测分析程序的总控程序。

输入：输入串 w 和文法 $G=(V, T, P, S)$ 的分析表 M ;

输出：如果 w 属于 $L(G)$ ，则输出 w 的最左推导，否则报告错误;

步骤：

1. 将栈底符号 $\#$ 和文法开始符号 S 压入栈中;
2. repeat
3. $X :=$ 当前栈顶符号;
4. $a :=$ 当前输入符号;
5. if $X \in T \cup \{\#\}$ then
6. if $X = a$ then
7. {if $X \neq \#$ then begin
8. 将 X 弹出栈;
9. 前移输入指针
10. end}



预测分析程序的总控程序

```
11.          else error
12.      else
13.          if  $M[X, a] = Y_1 Y_2 \dots Y_k$  then begin
14.              将 $X$ 弹出栈;
15.              依次将 $Y_k, \dots, Y_2, Y_1$ 压入栈;
16.              输出产生式 $X \rightarrow Y_1 Y_2 \dots Y_k$ 
17.          end
18.      else error
19.  until  $X = \#$ 
```




4.3.2 预测分析表的构造算法

算法4.6 预测分析表($LL(1)$ 分析表)的构造算法。

输入：文法 G ;

输出：分析表 M ;

步骤：

1. 对 G 中的任意一个产生式 $A \rightarrow \alpha$, 执行第2步和第3步;
2. for $\forall a \in \text{FIRST}(\alpha)$, 将 $A \rightarrow \alpha$ 填入 $M[A, a]$;
3. if $\varepsilon \in \text{FIRST}(\alpha)$ then
 - $\forall a \in \text{FOLLOW}(A)$, 将 $A \rightarrow \alpha$ 填入 $M[A, a]$;
 - if $\varepsilon \in \text{FIRST}(\alpha) \ \&\# \in \text{FOLLOW}(A)$ then
将 $A \rightarrow \alpha$ 填入 $M[A, \#]$;
4. 将所有无定义的 $M[A, b]$ 标上出错标志。

4.3.2 预测分析表的构造算法

$E \rightarrow TE'$ $E' \rightarrow +TE' | \epsilon$ $T \rightarrow FT'$ $T' \rightarrow *FT' | \epsilon$ $F \rightarrow (E) | id$

算法4.6 预测分析表(LL(1)分析表)的构造算法。

输入：文法G;

输出：分析表M;

步骤：

- 1. 对G中的任意一个产生式 $A \rightarrow \alpha$, 执行第2步和第3步;
- 2. for $\forall b \in \text{FIRST}(\alpha)$, 将 $A \rightarrow \alpha$ 填入 $M[A, b]$;
- 3. if $\epsilon \in \text{FIRST}(\alpha)$ then
 $\forall b \in \text{FOLLOW}(A)$, 将 $A \rightarrow \alpha$ 填入 $M[A, b]$;
 if $\epsilon \in \text{FIRST}(\alpha) \ \& \ \# \in \text{FOLLOW}(A)$ then
 将 $A \rightarrow \alpha$ 填入 $M[A, \#]$;
- 4. 将所有无定义的 $M[A, t]$ 标上出错标志。

$\text{FOLLOW}(E') = \{), \# \}$
 $\text{FOLLOW}(T') = \{ +,), \# \}$
 $\text{FIRST}(TE') = \{ (, id \}$ $\text{FIRST}(+TE') = \{ + \}$
 $\text{FIRST}(FT') = \{ (, id \}$ $\text{FIRST}(*FT') = \{ * \}$
 $\text{FIRST}((E)) = \{ (\}$ $\text{FIRST}(id) = \{ id \}$

非终结符	输入符号					
	id	+	*	(	)	#
E	$\rightarrow TE'$			$\rightarrow TE'$		
E'		$\rightarrow +TE'$			$\rightarrow \epsilon$	$\rightarrow \epsilon$
T	$\rightarrow FT'$			$\rightarrow FT'$		
T'		$\rightarrow \epsilon$	$\rightarrow *FT'$		$\rightarrow \epsilon$	$\rightarrow \epsilon$
F	$\rightarrow id$			$\rightarrow (E)$		



对输入串id+id*id进行分析的过程

(请大家在纸上同时画出语法树)

栈	输入缓冲区	输出
#E	id+id*id#	
#E'T	id+id*id#	$E \rightarrow TE'$
#E'T'F	id+id*id#	$T \rightarrow FT'$
#E'T'id	id+id*id#	$F \rightarrow id$
#E'T'	+id*id#	
#E'	+id*id#	$T' \rightarrow \epsilon$
#E'T+	+id*id#	$E' \rightarrow +TE'$
#E'T	id*id#	

非终结符	输入符号					
	id	+	*	(	)	#
E	$\rightarrow TE'$			$\rightarrow TE'$		
E'		$\rightarrow +TE'$			$\rightarrow \epsilon$	$\rightarrow \epsilon$
T	$\rightarrow FT'$			$\rightarrow FT'$		
T'		$\rightarrow \epsilon$	$\rightarrow *FT'$		$\rightarrow \epsilon$	$\rightarrow \epsilon$
F	$\rightarrow id$			$\rightarrow (E)$		



对输入串id+id*id进行分析的过程

#E'T	id*id#	
#E'T'F	id*id#	T→FT'
#E'T'id	id*id#	F→id
#E'T'	*id#	
#E'T'F*	*id#	T'→*FT'
#E'T'F	id#	
#E'T'id	id#	F→id
#E'T'	#	
#E'	#	T'→ε
#	#	E'→ε

非终结符	输入符号					
	id	+	*	(	)	#
E	→TE'			→TE'		
E'		→+TE'			→ε	→ε
T	→FT'			→FT'		
T'		→ε	→*FT'		→ε	→ε
F	→id			→(E)		

输出的产生式序列形成了最左推导对应的分析树



预测分析法的实现步骤

1. 构造文法
2. 改造文法：消除二义性、消除左递归、提取左因子
3. 求每个候选式的FIRST集和变量的FOLLOW集
4. 检查是不是 LL(1) 文法
若不是 LL(1),说明文法的复杂性超过自顶向下方法的分析能力, 需要附加新的 “信息”
5. 构造预测分析表
6. 实现预测分析器



4.3.3 预测分析中错误的处理

- 对语法变量 A ，如果 $M[A,a]$ 无定义，并且 a 属于 $FOLLOW(A)$ ，则增加 $M[A,a]$ 为“同步点” (synch), 同步记号选择方法如下：
 - 把 $FOLLOW(A)$ 的所有符号放入语法变量 A 的同步记号集合中。
 - 把 A 后高层结构的开始符号加到低层结构的同步记号集合中。
 - 把 $FIRST(A)$ 的符号加入 A 的同步记号集合。
 - 如果语法变量可以产生空串，若出错时栈顶是这样的语法变量，则可以使用产生空串的产生式。
 - 如果符号在栈顶而不能匹配，则弹出此符号。

在编译过程中，如果遇到错误应该()。

- ☐ A 对错误在局部范围内进行纠正，继续向下分析
- ☐ B 当发现错误时，跳过错误所在的语法单位继续分析下去
- ☐ C 当发现错误时立即停止编译，待用户改正错误后再继续编译



4.4 递归下降分析法 — 一个设想

1. 对应每个变量设置一个处理子程序:

$$A \rightarrow X_1 X_2 \dots X_k \dots X_n$$

(1) 当遇到 X_k 是终极符号时直接进行匹配;

(2) 当遇到 X_k 是语法变量时就调用 X 对应的处理子程序.

2. 要求处理子程序是可以递归调用的

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' | \varepsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' | \varepsilon$$

$$F \rightarrow (E) | id$$



4.4.1 递归下降分析法的基本思想

例4.14 对于产生式 $E' \rightarrow +TE'$ ，与 E' 对应的子程序可以按如下方式来编写：

```
procedure  $E'$ 
begin
    match('+');
     $T$ ;          /*调用识别 $T$ 的过程*/
     $E'$           /*调用识别 $E'$ 的过程*/
end;
```



4.4.1 递归下降分析法的基本思想

其中，服务子程序 $match$ 用来匹配当前的输入记号，其代码为：

```
procedure match(t:token);  
  
begin  
    if lookhead=t then  
        lookhead:=nexttoken;  
    else error          /*调用出错处理程序*/  
end;
```



4.4.2 语法图和递归子程序法

- 状态转换图（语法图）是非常有用的设计工具
- 语法分析器和词法分析器的状态转换图不同
 - 每个非终结符对应一个状态转换图，边上的标记是记号和非终结符
 - 记号上的转换意味着如果该记号是下一个输入符号，就应进行转换
 - 非终结符A上的转换是对与A对应的过程的调用



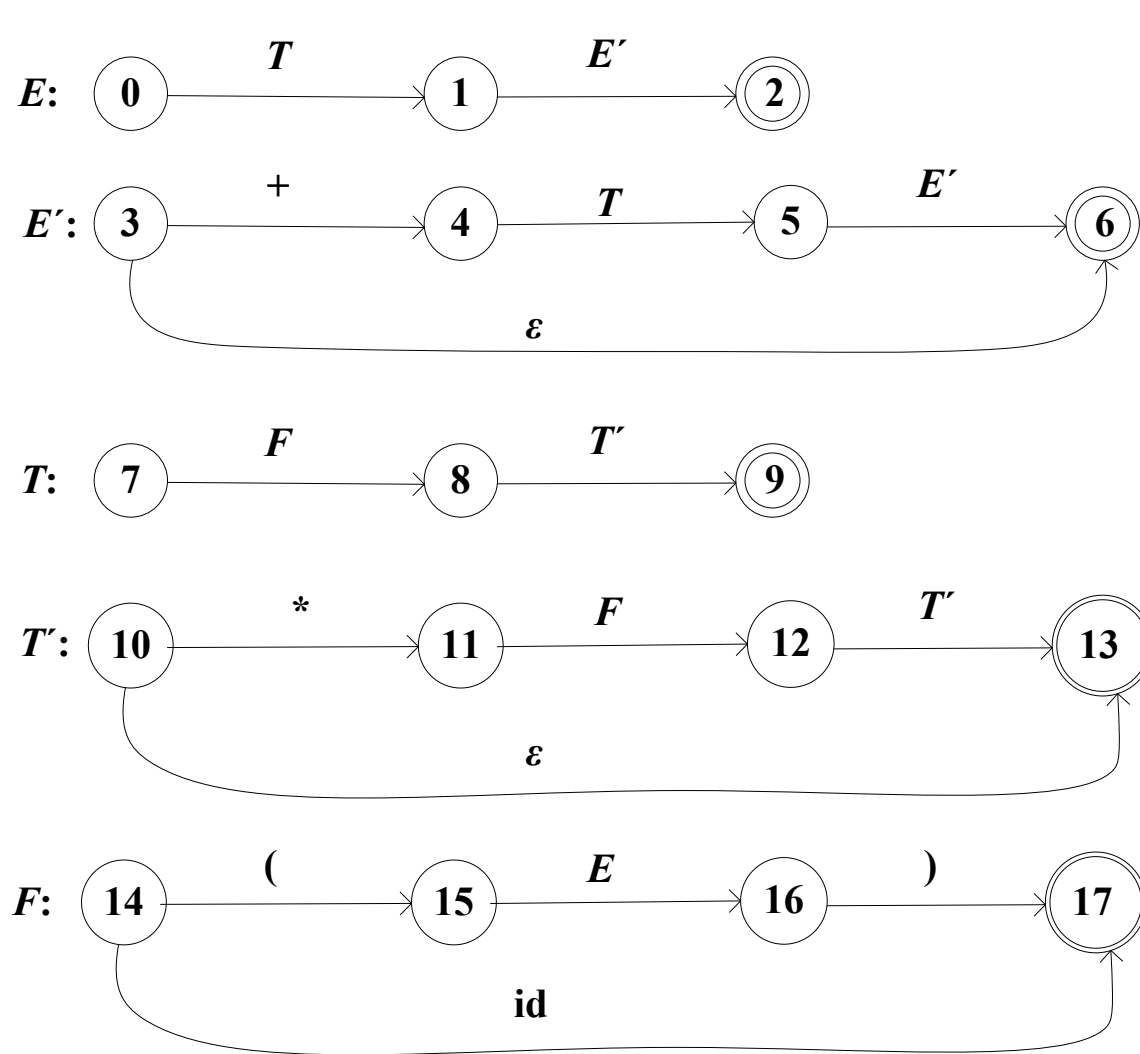
4.4.2 语法图和递归子程序法

□从文法构造语法图，对每个非终结符A执行如下操作

- 创建一个开始状态和一个终止状态（返回状态）
- 对每个产生式 $A \rightarrow X_1 X_2 \dots X_n$ ，创建一条从开始状态到终止状态的路径，边上的标记分别为 $X_1, X_2, \dots, X_n$



例4.15 简单表达式文法的语法图



$E \rightarrow TE'$

$E' \rightarrow +TE' | \epsilon$

$T \rightarrow FT'$

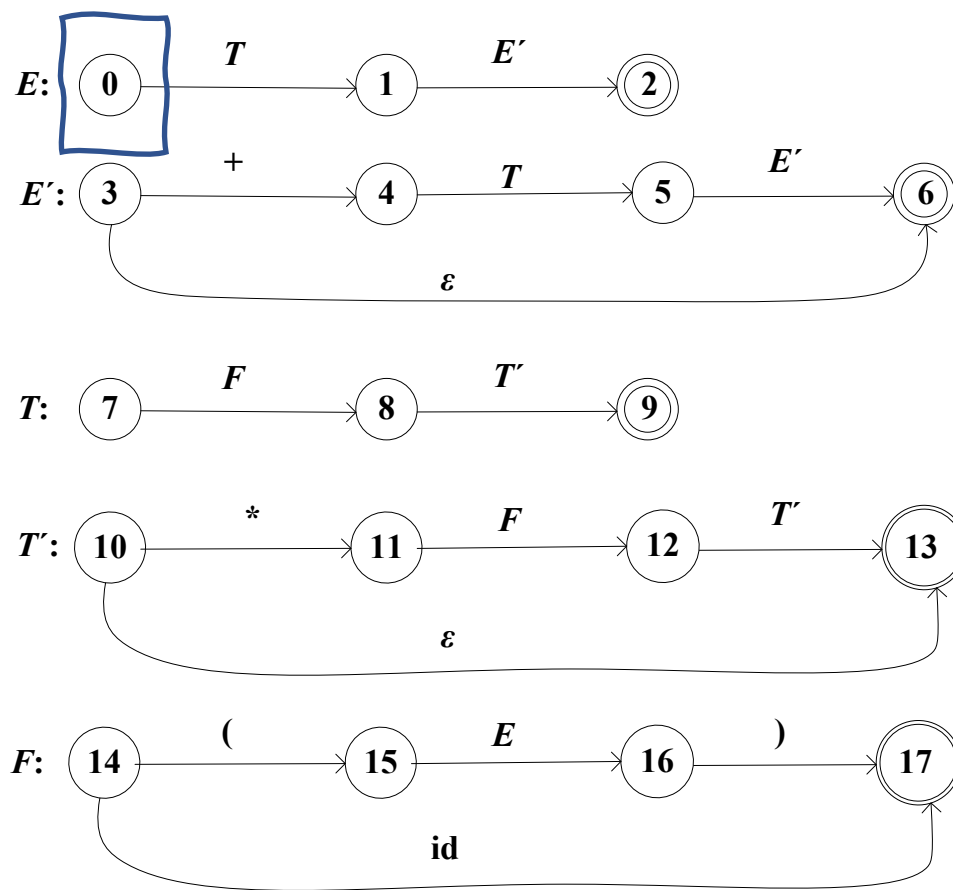
$T' \rightarrow *FT' | \epsilon$

$F \rightarrow (E) | id$



4.4.3基于语法图的语法分析器工作方式

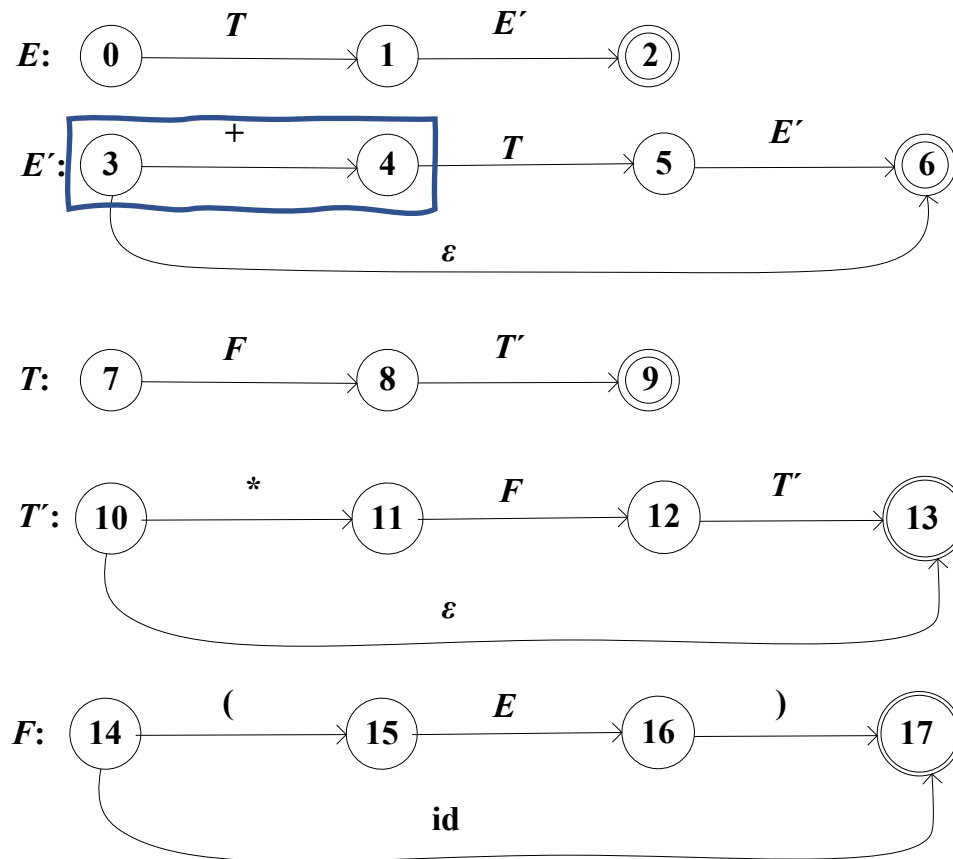
□初始时，分析器进入状态图的开始状态，输入指针指向输入符号串的第一个符号。





4.4.3基于语法图的语法分析器工作方式

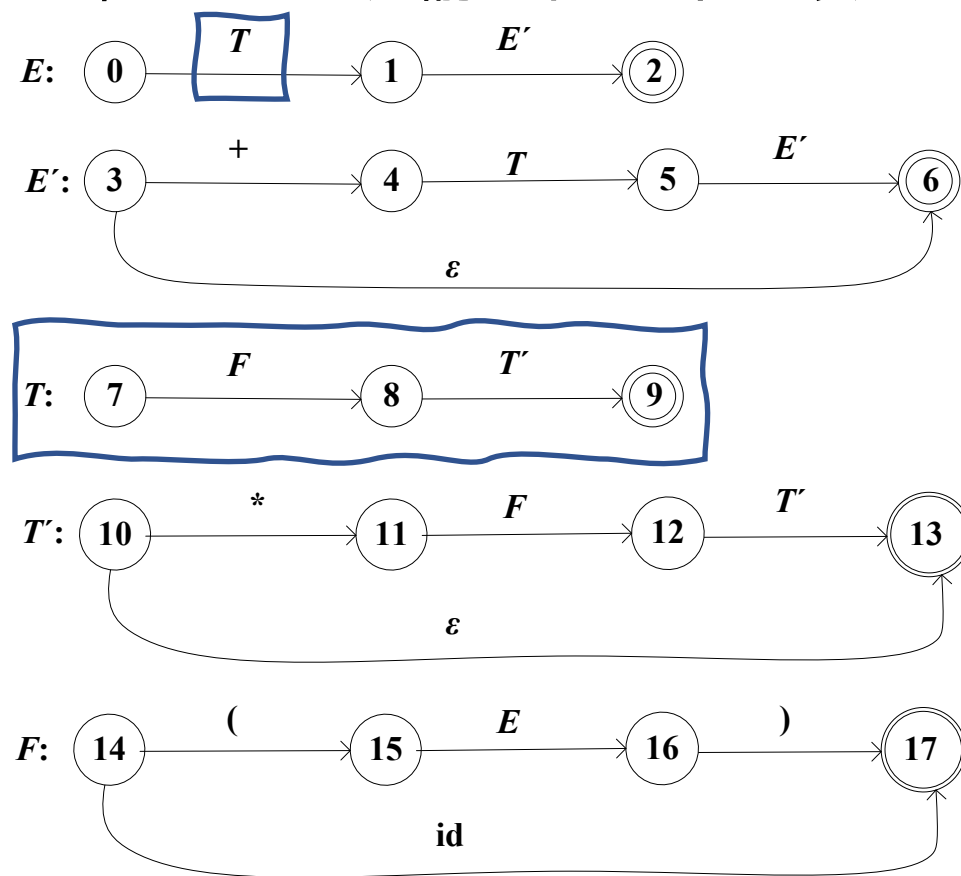
□如果经过一些动作后，它进入状态 s ，且从状态 s 到状态 t 的边上标记了终结符 a ，此时下一个输入符又正好是 a ，则分析器将输入指针向右移动一位，并进入状态 t 。





4.4.3基于语法图的语法分析器工作方式

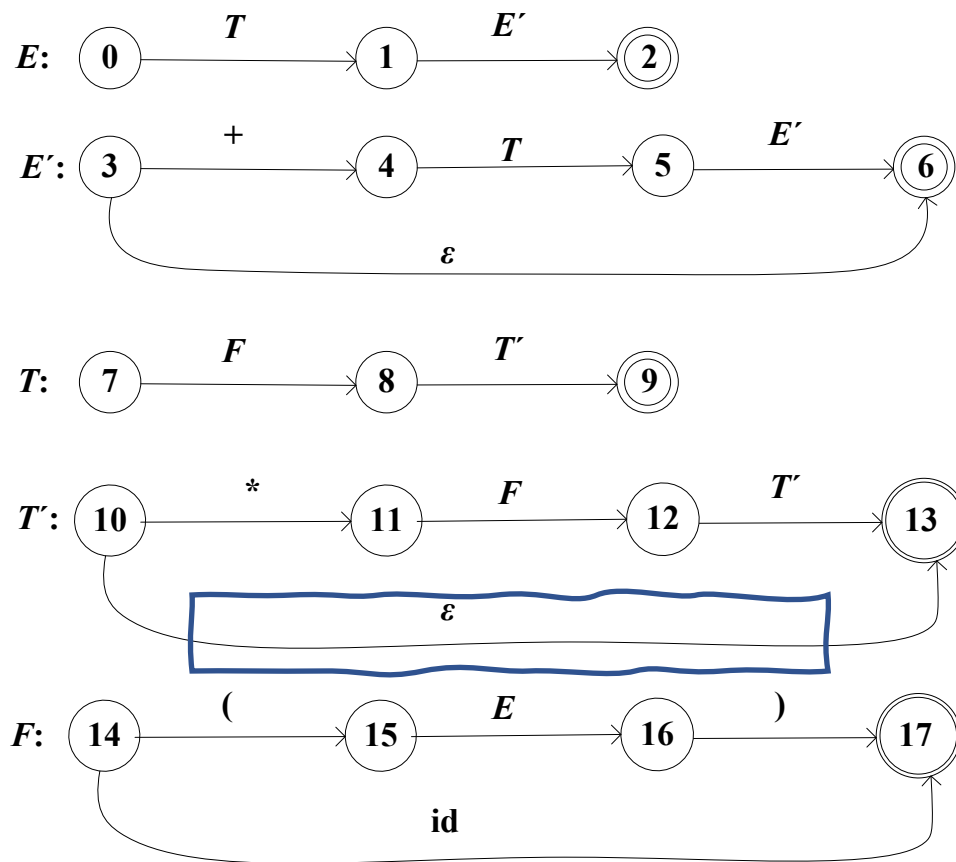
□另一方面，如果边上标记的是非终结符A，则分析器进入A的初始状态，但不移动输入指针。一旦到达A的终态，则立刻进入状态t，事实上，分析器从状态s转移到状态t时，它已经从输入符号串“读”了A（调用A对应的过程）。





4.4.3基于语法图的语法分析器工作方式

□如果从s到t有一条标记为 ϵ 的边，那么分析器从状态s直接进入状态t而不移动输入指针。





4.4.4 语法图的化简与实现

- (1) 左因子提取 将形如 $A \rightarrow YX|YZ$ 的产生式替换为 $A \rightarrow Y(X|Z)$;
- (2) 右因子提取 将形如 $A \rightarrow YX|ZX$ 的产生式替换为 $A \rightarrow (Y|Z)X$;
- (3) 尾递归消除 将形如 $X \rightarrow YX|Z$ 的产生式替换为 $X \rightarrow Y^*Z$ 。

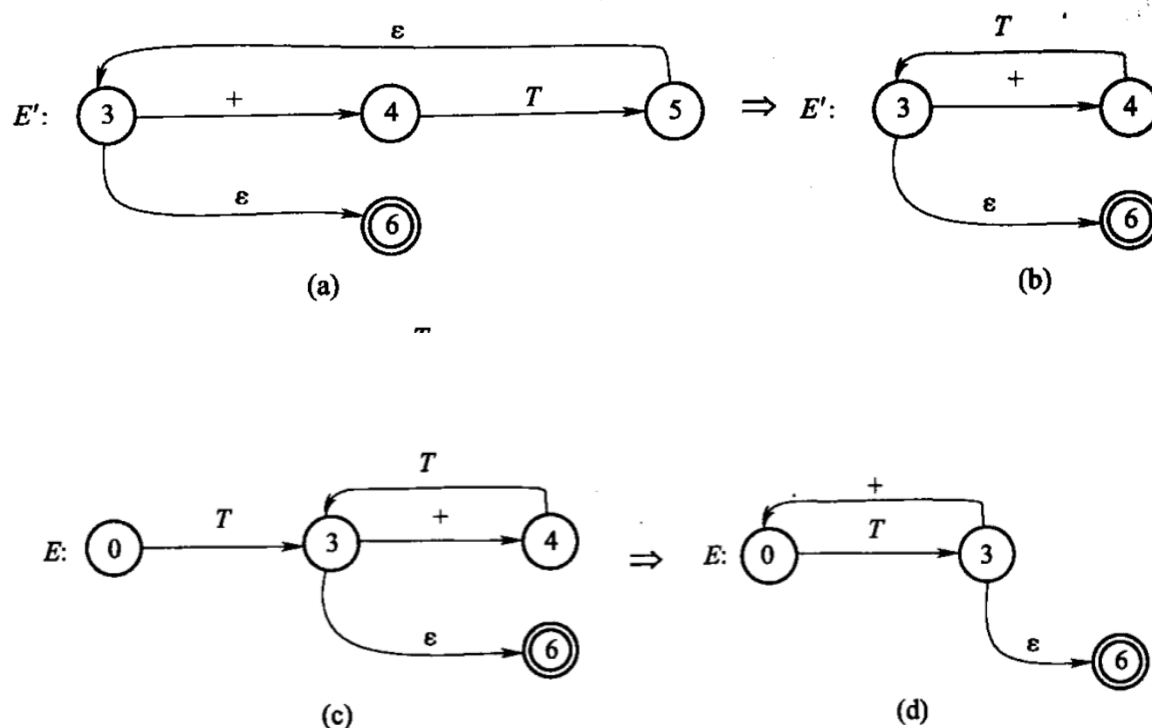
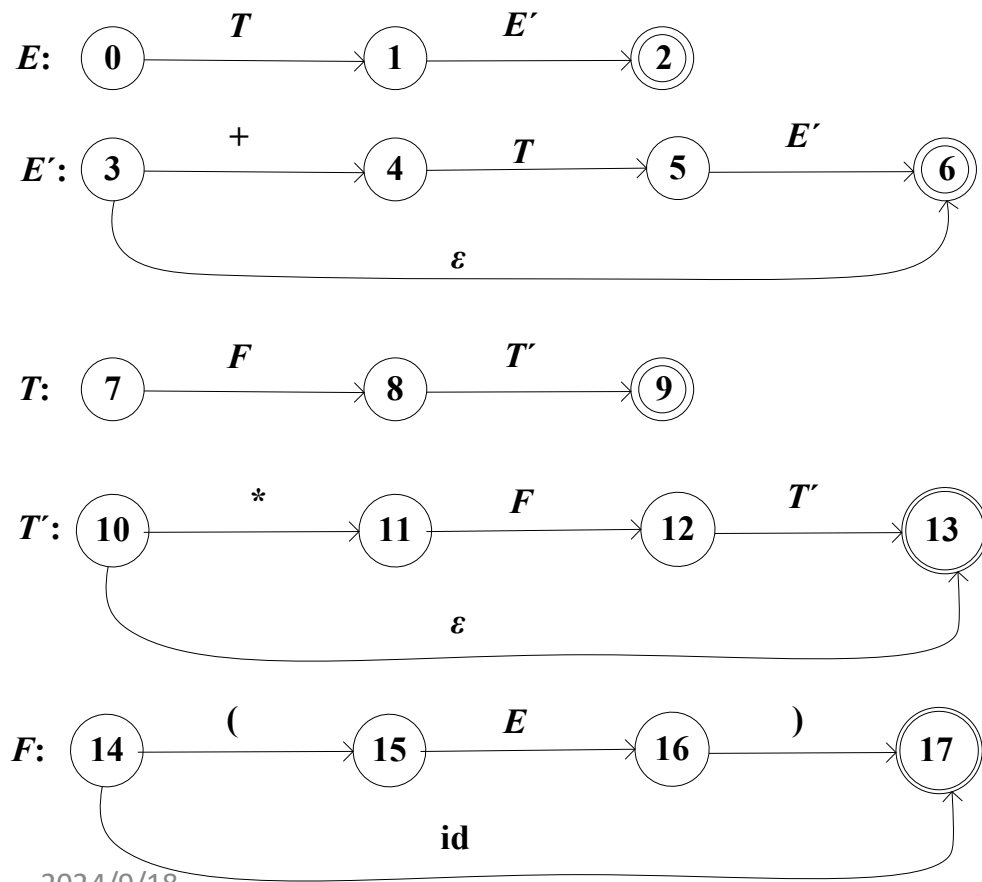


图4.6算术表达式的简化语法图



4.4.4 语法图的化简与实现

- (1) 左因子提取 将形如 $A \rightarrow YX|YZ$ 的产生式替换为 $A \rightarrow Y(X|Z)$;
- (2) 右因子提取 将形如 $A \rightarrow YX|ZX$ 的产生式替换为 $A \rightarrow (Y|Z)X$;
- (3) 尾递归消除 将形如 $X \rightarrow YX|Z$ 的产生式替换为 $X \rightarrow Y^*Z$ 。

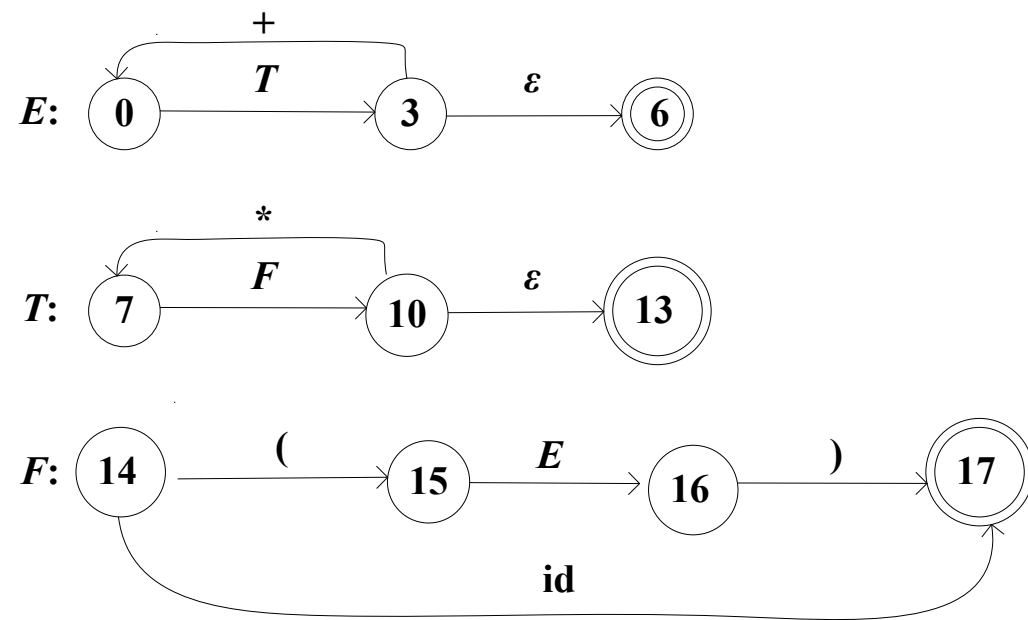
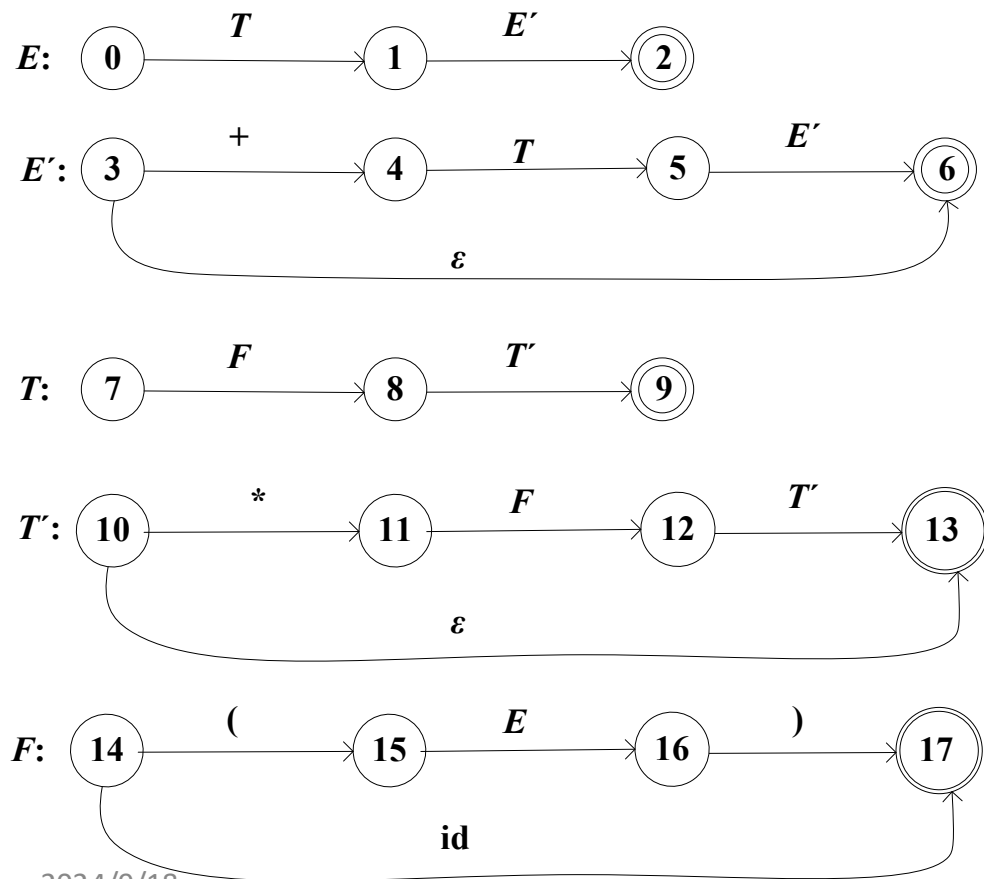


图4.6算术表达式的简化语法图



例4.16 简单算术表达式的语法分析器

□ E 的子程序($E \rightarrow T(+T)^*$)

procedure E;

begin

T; T的过程调用

while lookahead='+' do

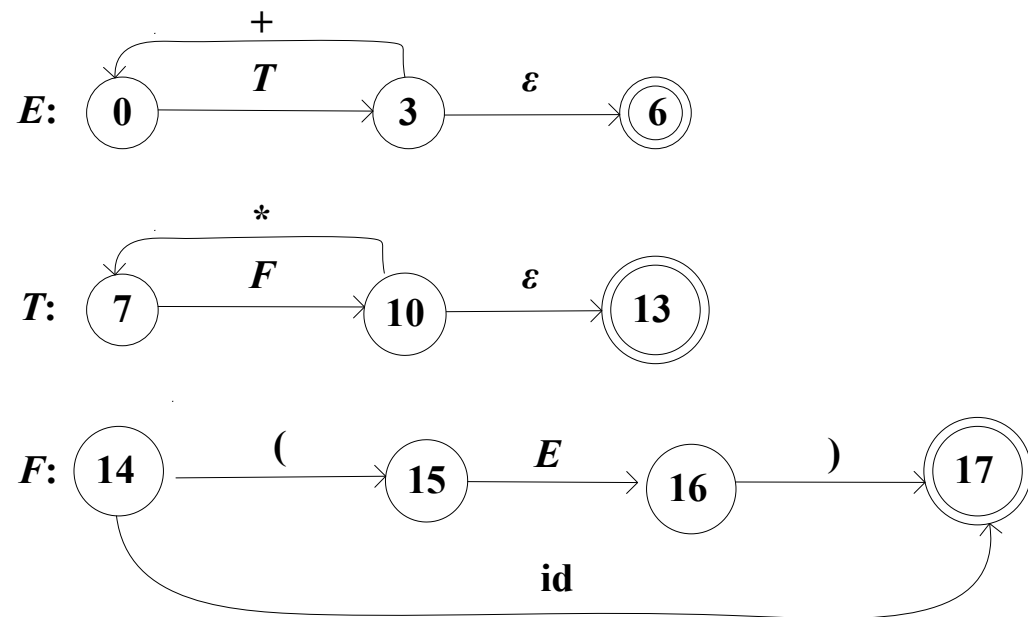
begin 当前符号等于+时

match('+'); 处理终结符+

T T的过程调用

end

end; lookahead: 当前符号





T 的子程序($T \rightarrow F(*F)^*$)

procedure T;

begin

F; F的过程调用

while lookahead='*' then

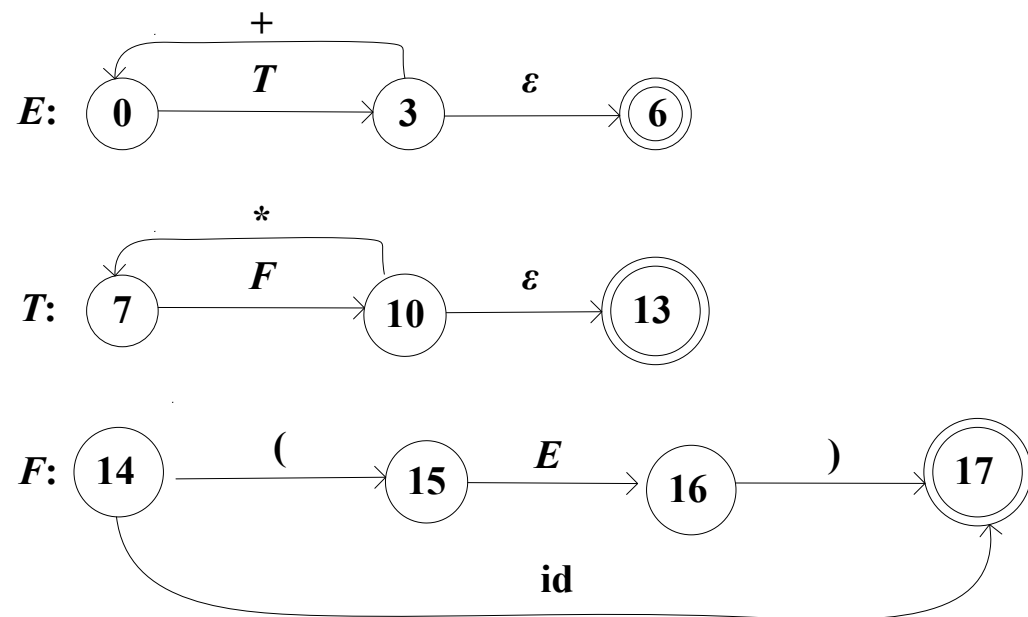
begin 当前符号等于*时

match('*'); 处理终结符*

F F的递归调用

end

end;





F 的子程序($F \rightarrow (E)|id$)

procedure F;

begin

if lookahead='(' then

begin

match('(');

E;

match(')');

end

else if lookahead=id then

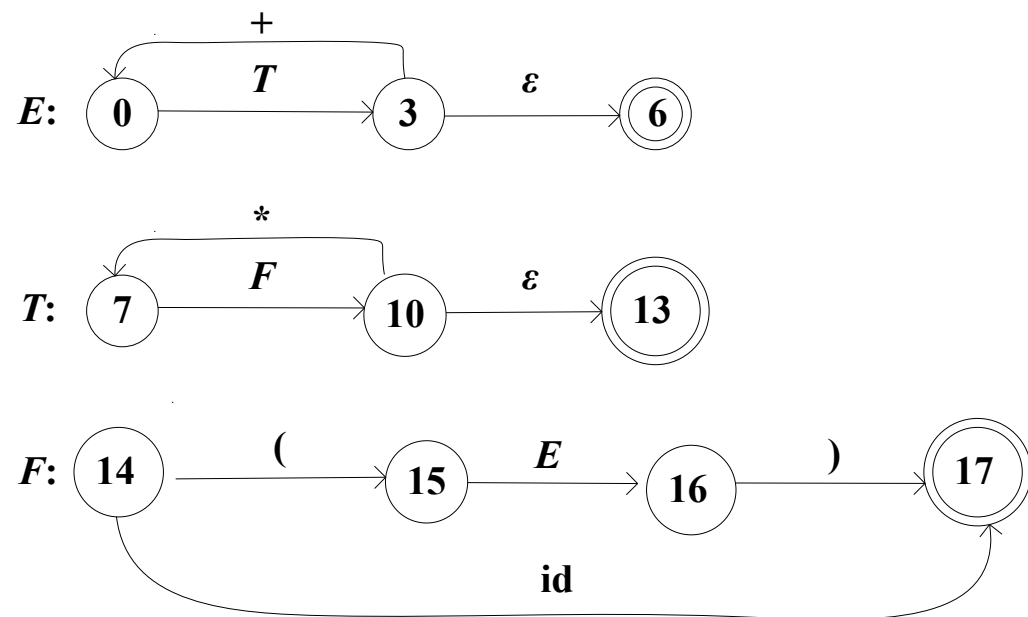
match(id);

else error

end

当前符号等于 (
 处理终结符 (
 E 的递归调用
 处理终结符)

处理终结符id
 出错处理





主程序

```
begin  
  lookahead:=nexttoken;  
  E  
end
```

调词法分析程序
E 的过程调用

服务子程序

```
procedure match(t:token);  
begin  
  if lookahead=t then  
    lookahead:=nexttoken  
  else error  
end;
```

出错处理程序



4.4.5 递归子程序法的实现步骤

- 1) 构造文法;
- 2) 改造文法: 消除二义性、消除左递归、提取左因子;
- 3) 求每个候选式的FIRST集和语法变量的FOLLOW集;
- 4) 检查 G 是不是 $LL(1)$ 文法, 若 G 不是 $LL(1)$ 文法, 说明文法 G 的复杂性超过了自顶向下方法的分析能力, 需要附加新的“信息”;
- 5) 按照 $LL(1)$ 文法画语法图;
- 6) 化简语法图;
- 7) 按照语法图为每个语法变量设置一个子程序。



递归子程序法的优缺点分析

□优点:

- 1) 直观、简单、可读性好
- 2) 便于扩充

□缺点:

- 1) 递归算法的实现效率低
- 2) 处理能力相对有限
- 3) 通用性差, 难以自动生成

□从递归子程序法及FIRST与FOLLOW集看如何进一步用好当前的输入符号?



本章小结

1. 自顶向下分析法和自底向上分析法分别寻找输入串的最左推导和最左归约
2. 自顶向下分析会遇到二义性问题、回溯问题、左递归引起的无穷推导问题，需对文法进行改造：消除二义性、消除左递归、提取公共左因子
3. $LL(1)$ 文法是一类可以进行确定分析的文法，利用FIRST集和FOLLOW集可以判定某个上下文无关文法是否为 $LL(1)$ 文法



本章小结

4. $LL(1)$ 文法可以用 $LL(1)$ 分析法进行分析。
5. 递归下降分析法根据各个候选式的结构为每个非终结符编写一个子程序。
6. 使用语法图可以方便地进行递归子程序的设计。